
GOAL-ORIENTED INFLUENCE DATA ACQUISITION

Weichi Yao

The Michigan Institute for Data Science
University of Michigan
Ann Arbor
weichiy@umich.edu

Bianca Dumitrascu

Department of Statistics
Columbia University
New York
bmd2151@columbia.edu

Bryan R. Goldsmith

Department of Chemical Engineering
University of Michigan
Ann Arbor
bgoldsm@umich.edu

Yixin Wang

Department of Statistics
University of Michigan
Ann Arbor
yixinw@umich.edu

ABSTRACT

We study iterative data acquisition in scientific and engineering problems where evaluating the objective—via experiments, simulations, or model training—is expensive. Existing Bayesian optimization and deep active learning methods typically rely on surrogate model uncertainty, which is often poorly calibrated and costly to estimate for modern neural networks. We propose Goal-Oriented Influence data acquisition, a unified framework that selects new inputs by maximizing their influence on a user-specified goal functional of the model parameters, such as test loss, predictive entropy, or the value of an optimizer-recommended design. Using first-order influence functions, we derive a tractable acquisition rule that couples the gradient of the goal, the curvature of the training loss, and the candidate’s parameter sensitivity, and we approximate unknown labels and goals with a Jackknife deep ensemble and efficient inverse-Hessian-vector products. Under exponential-family models, we show that the resulting criterion recovers the leverage structure of predictive-entropy minimization while additionally calibrating by goal alignment and prediction bias, thereby implicitly capturing uncertainty without maintaining a full Bayesian posterior. Across image and text classification, noisy global optimization benchmarks, and neural-network hyperparameter tuning, Goal-Oriented Influence consistently achieves target performance with substantially fewer labeled samples or function evaluations than uncertainty-based active learning and Gaussian-process Bayesian optimization baselines.

Keywords Deep learning · Active acquisition · Bayesian optimization · Active learning · Influence function

1 Introduction

Many scientific tasks can be formulated as an optimization problem. The objective function often involves complex components that are expensive to evaluate or label. For example, a central task in materials science is to discover materials with exceptional properties [77, 87]. These properties can be formulated as functions of elemental composition and atomic arrangement, and the goal is to optimize these functions over the vast design space of possible materials. The synthesis of the material and experimental testing to evaluate the properties is often time-consuming and expensive.

In a related setting, researchers seek to characterize the response of a biological system to stimuli [112]. A typical workflow involves training a neural network on available data to predict the response to a stimulus and then applying the trained model to the stimulus of interest to obtain predicted responses. The goal is to minimize the discrepancy between the predicted and true responses. The key challenge of this workflow is that the ground-truth responses require complex and costly laboratory experiments.

Another illustrative example is hyperparameter tuning for complex models, such as deep neural networks [124]. Hyperparameters specify the network architecture (e.g., width, depth) and the training procedure (e.g., learning rate, weight decay), and they have a massive influence on the model training performance. Therefore, scientists must carefully tune the hyperparameters to realize the full capacity of the model architecture—in other words, optimizing the model performance in terms of minimizing the validation loss (label). The search for the best hyperparameters inevitably requires training the network with multiple sets of hyperparameters and validating the performance on a held-out dataset, which is computationally expensive.

When data acquisition is expensive, it is crucial to minimize the number of evaluations needed to optimize the *goal* objective function while efficiently exploring arbitrary design spaces. A natural strategy is to acquire observations *iteratively*—starting from an initial dataset $\mathcal{D}$ (which can be empty), at each iteration, first fit a model $\mathcal{M}$ to the current data, which reflects the most up-to-date belief; then evaluate or label the next data point x expected to be most *valuable* for achieving the project goal, usually through the maximization of some acquisition function a that quantifies the anticipated benefit of labeling x . See Algorithm 1.

Algorithm 1 Iterative data acquisition

Input: The initial data set $\mathcal{D}$

- 1: **repeat**
 - 2: Fit a model $\mathcal{M}$ on $\mathcal{D}$
 - 3: Select $x \leftarrow \arg \min_x a_{\mathcal{M}}(x; \mathcal{D})$
 - 4: Query $y \leftarrow \text{OBSERVE}(x)$
 - 5: Augment $\mathcal{D} \leftarrow \mathcal{D} \cup \{(x, y)\}$
 - 6: **until** termination condition is met {e.g., budget exhausted or desired goal achieved}
-

Iterative data acquisition algorithms designed to solve these problems usually choose $\mathcal{M}$ to be neural networks due to their strong learning capacities, and the data selection often relies on some measure of uncertainty. For example, Bayesian optimization algorithms, commonly used for material design or hyperparameter tuning, set $\mathcal{M}$ in Algorithm 1 to be a Bayesian Neural Network [75] and constructs an acquisition function based on the inferred posterior predictive distribution and its corresponding uncertainties [32, 120]. For biology stimuli-response studies, classic iterative data acquisition approaches are Active Learning algorithms, in which the model $\mathcal{M}$ is set to be deep neural networks whose parameters are to be optimized to learn the relationship between the stimuli and system responses, and the most widely used acquisition criterion is uncertainty-based [100, 97, 92, 72].

Uncertainty quantification in deep learning models remains a major challenge, particularly for high-dimensional inputs. Exact inference is intractable, and approximate methods are often computationally expensive and rely on large training sets [4]. Despite much progress, obtaining reliable uncertainty estimates in deep neural networks is still challenging, which compromises the effectiveness of data acquisition strategies [108, 75, 4].

1.1 Main Idea

In this work, we propose “Goal-Oriented Influence-Based Data Acquisition,” a class of effective iterative data acquisition methods that do not rely on external model uncertainty estimates. It selects data points based on the *influence* of the candidate data point on the target *goal objective function* defined by the scientific application—that is, it is able to maximize an objective while minimizing data acquisition efforts. We will show that this formulation implicitly calibrates model uncertainty [27, 41, 9, 46, 3] and additionally allows model bias quantification, which help strike a balance between *exploration* and *exploitation*. When compared to existing active learning and Bayesian Optimization methods, Goal-Oriented Influence achieves better downstream task performance in predictive learning and function optimization with fewer labeled samples/function evaluations.

Formally, let (x, y) be the observed feature-label pair with $x \in \mathcal{X}$ and $y \in \mathcal{Y}$, and $p_0(y | x)$ denote the true conditional distribution. Assume obtaining x is free, while getting observations y given x or sampling from $p_0(y | x)$ is costly. We consider $\mathcal{M}_\theta$ a supervised model $p_\theta(y | x)$ with parameters θ . We fit the model with a loss function $\ell(\theta; (x, y)) = -\log p_\theta(y | x)$.

The goal objective function $\mathcal{G}$ can be flexibly defined for various scientific tasks that can be formulated into an optimization problem. Without loss of generality, the following discussion targets the minimization of $\mathcal{G}$. We consider the goal objective function $\mathcal{G}$ as a function of θ , which parametrizes and reflects the best knowledge of the unknown distribution p_0 .

Goal-Oriented Influence iterative data acquisition alternates between two steps:

1. At each step, we fit a model $p_\theta(y | x)$ with model parameters θ by minimizing the empirical risk function on the existing data set $\mathcal{D}$

$$\theta(\mathcal{D}) = \arg \min_{\theta} \frac{1}{|\mathcal{D}|} \sum_{(u,v) \in \mathcal{D}} \ell(\theta; (u,v)) \quad (1)$$

2. We choose to label the next data point that maximizes the expected influence $\mathcal{I}$ of any candidate x on the goal objective function $\mathcal{G}$

$$x_{\text{next}} = \arg \max_x \mathbb{E}_y [\mathcal{I} \circ \mathcal{G}(\theta(\mathcal{D} \cup (x,y)))] \quad (2)$$

where the influence function computes the instantaneous rate at which upweighting x gives the expected maximum reduction in $\mathcal{G}$

$$E_y [\mathcal{I} \circ \mathcal{G}(\theta(\mathcal{D} \cup (x,y)))] := \left. \frac{\partial}{\partial \epsilon} [E_y [\mathcal{G}(\theta^\epsilon(\mathcal{D} \cup \{(x,y)\}))]] \right|_{\epsilon=0} \quad (3)$$

In this formulation, $\theta^\epsilon(\mathcal{D} \cup \{(x,y)\})$ is the parameter estimate after x is added to the current dataset $\mathcal{D}$ with weight ϵ , and the output of any candidate x is approximated using resampling techniques such as Jackknife [116].

1.2 Related Works

1.2.1 Iterative data acquisition for optimization

The iterative data acquisition strategy in Algorithm 1 is the standard template for solving expensive black-box (global) optimization problems in which the objective has no closed form and gradients are unavailable [15, 90, 91, 39]. The most prevailing machine learning class of methods is *Bayesian optimization* (BO) [102, 32, 120], which maintains a probabilistic surrogate $\mathcal{M}$ for the unknown objective and selects new inputs by optimizing an acquisition function that encodes an exploration–exploitation trade-off. In classical BO, $\mathcal{M}$ is often a Gaussian process (GP) [89], and popular acquisitions include Probability of Improvement (PI) [114, 67], Expected Improvement (EI) [57, 80], Upper Confidence Bound (UCB) [109], Entropy Search (ES) [50, 36], Max-value Entropy Search (MES) [122], and Knowledge Gradient (KG) [33, 34, 98, 123, 124].

Despite considerable empirical success, BO has practical limitations [102, 32, 39, 120]. First, GP-based surrogates require forming and factorizing an $n \times n$ kernel matrix each iteration, yielding $\mathcal{O}(n^2)$ memory and $\mathcal{O}(n^3)$ time per update [89, 39], which becomes prohibitive as n grows. To improve scalability or cope with high dimensions, recent work employs structured/high-dimensional GP constructions or low-dimensional embeddings [121, 82, 81, 103], and localized search such as TuRBO [28, 29]; these approaches reduce cost but often impose restrictive model assumptions, add posterior-maintenance overhead, and can be sensitive to miscalibration [13, 120]. Second, replacing GPs with alternatives—for example, density/tree-based models [11, 55] or Bayesian neural surrogates [107, 108]—improves scalability but can degrade uncertainty calibration or robustness in some regimes [13, 75]. Third, many acquisitions require expectations under the surrogate’s posterior predictive; a fully Bayesian treatment (integrating over hyperparameters and computing acquisition expectations exactly) is rarely tractable. In practice, one uses plug-in hyperparameters [57, 28] and/or Monte-Carlo estimators [108, 107, 123, 124, 59], which respectively sacrifice proper marginalization and add computational overhead and estimator variance [16, 102, 13, 75, 39].

Our positioning Motivated by these drawbacks, we pursue an alternative that does *not* rely on computationally intensive posterior inference. Our *goal-oriented influence* criterion scores candidates by the first-order impact that upweighting them would have on a task-level goal, yielding an exploration-aware, bias-focused improvement alternative approach. Under exponential-family modeling, Goal-Oriented Influence preserves the same curvature/variance factors that underlie predictive-entropy acquisition (uncertainty awareness) while calibrating them by goal alignment and prediction bias (exploitation focus). As later sections show, Goal-Oriented Influence mirrors the *exploitation* component of Bayesian optimization—without maintaining a posterior—offering a practical alternative when Bayesian updates are expensive or poorly calibrated [102, 120, 39].

1.2.2 Iterative data acquisition for learning

Another class of algorithms that adopt the iterative data acquisition paradigm under high labeling cost is *active learning* (AL). AL is designed for supervised learning settings where acquiring labeled training data for a predictive model is expensive, and the learner must decide which inputs to query next. The biological stimuli–response study is one example where AL can help obtain predictive models with high precision at limited experimental cost, by carefully

selecting which stimuli to probe to obtain system responses used for training the model. In particular, we focus on AL for deep learning (DL) predictive models, which have achieved unprecedented breakthroughs across various fields because of their strong representation and function-approximation capabilities. In the context of deep learning, AL algorithms mostly focus on classification tasks for image and text data [92, 97, 72] and have also been covered in more general AL surveys [100].

In pool-based deep AL, an *acquisition function* assigns a utility score to each unlabeled point, and the learner queries the highest-scoring points at each iteration. Most acquisition criteria designed for deep neural networks can be broadly grouped into three families [92, 72]: (i) uncertainty-based methods, which query points the model is most uncertain about [51, 117, 14, 74, 118, 88, 38, 128, 68, 10, 105, 115, 61, 111, 63, 126]; (ii) representation-based methods, which select points that are prototypical or diverse in a learned feature space [18, 99, 131, 71, 48, 21, 43, 60, 56, 73, 85]; and (iii) hybrid methods that combine uncertainty information with diversity or representativeness in feature or gradient space [26, 129, 132, 106, 6, 104, 19, 42, 53, 40, 5, 95]. The latter two families are especially common in batch-mode acquisition, where one aims to select a set of informative yet non-redundant points in each AL round.

Among these, *uncertainty-based* criteria are by far the most widely used, both in classical AL and in deep AL [100, 97]. They are often instantiated via confidence-, margin-, or entropy-based scoring [117, 96, 93, 14, 74], Bayesian mutual-information objectives such as BALD and its batch extensions [51, 38, 111, 63], or expected-model-change-style criteria that aim to approximate loss reduction [101, 94, 35, 115, 127]. In deep learning, these strategies typically rely on approximate Bayesian neural networks (e.g., Monte Carlo dropout) [110, 37, 38] or deep ensembles [68, 10], and they have also been combined with discriminative or energy-based scoring functions [10, 61, 126]. However, predictive probabilities from modern neural networks are often poorly calibrated—deep models are known to be over-confident even when wrong or under distribution shift [44, 1, 97]. Moreover, Bayesian neural networks require approximate inference that is computationally demanding and whose quality can degrade in the small-data regimes characteristic of early AL rounds [4]. As noted in recent surveys, obtaining reliable and well-calibrated uncertainty estimates in deep models remains an open challenge, which in turn limits the robustness of uncertainty-driven deep AL [97, 92, 72].

Our positioning By contrast, in this work we build on *influence functions* as an alternative acquisition criterion. Instead of explicitly modeling predictive uncertainty, influence-based methods approximate how upweighting a candidate point would affect the trained model or a downstream evaluation objective, without retraining from scratch [65]. By adopting influence scores as the basis for query selection, Goal-Oriented Influence avoids the need for calibrated uncertainty estimates while still implicitly capturing aspects of uncertainty and prediction bias. As we show in Section 3, under an exponential family assumption Goal-Oriented Influence can be related to predictive entropy minimization: it implicitly calibrates uncertainty and explicitly incorporates a prediction-bias term, yielding an exploration-aware yet exploitation-focused acquisition rule.

1.2.3 Influence functions

Influence functions (IFs) originate in robust statistics as a way to quantify the infinitesimal effect of perturbing a data point on an estimator. Classical work by Cook and colleagues developed diagnostics for detecting influential observations and local influence in regression—covering single-point and grouped/structured perturbations, residual-based assessment, and empirical IF characterizations [22, 24, 23]. Building on these ideas, the deep-learning community reintroduced IFs for complex, nonconvex models: [65] showed how first-order IFs can approximate the effect of up-weighting or removing a training example on a black-box prediction, enabling practical tools for debugging, dataset curation, and attribution of test errors to specific training points. Subsequent work examined group-level effects [64], scalable approximations [45], higher-order/group IFs [7, 8], and uncertainty quantification via higher-order IF/Jackknife variants [3]. IF-based analyses have also been used to uncover data artifacts that models exploit [47], quantify memorization and long-tail phenomena [30], and probe representational bias [17].

Most IF-based work in deep learning is retrospective: given a fixed, labeled training set (and often fixed test points), IFs explain or diagnose existing predictions. By contrast, in iterative data acquisition the candidate’s label is *unknown* at selection time, so one must reason about the *expected* influence under $p_0(y | x)$. To our best knowledge, existing works that use the influence function for iterative data acquisition are under an active learning setup. The first line of works proposed to select data point iteratively based on the IFs of selected utility functions [127, 76, 125]. The utility functions can be understood as the “goal” of the learning. They considered the utility functions to be the training loss on an additional test dataset with known labels or prediction entropy which only rely on current training information p_θ . When the labels in the test dataset are unknown, they rely on approximation using posterior or p_θ purely based on the current training information, which is similarly done for the unknown label of the candidate data point before selection. Another recent work [119], instead of directly computing the influence of the candidate data point on test loss where the labels are unknown, it upper-bounds the test loss and selects the data point based on gradient norm of the loss on the

candidate, which avoids any direct evaluation of the test performance. When it comes to terms that involve the unknown candidate output, again it is approximated with the current estimated model p_θ purely based on training information.

Our positioning In this work, we study the influence function of the candidate data point on the goal function flexibly defined by a wide range of tasks of interest, for example, which can be used for iteratively solving a global optimization problem on a black-box complex function or acquiring labels to optimize performance on a unlabeled test dataset under an active learning setup. The unknown quantities are approximated by a separate ensemble model using the resampling techniques such as Jackknife, instead of relying purely on training information.

The rest of the paper is organized as follows: Section 2 presents the Goal-Oriented Influence iterative data acquisition algorithm in full detail. In Section 3, we derive the Goal-Oriented Influence under the exponential families distribution assumption. We describe the close connection between Goal-Oriented Influence and predictive entropy minimization in Bayesian generalized linear models, which suggests that Goal-Oriented Influence implicitly calibrates model uncertainty. We also show that Goal-Oriented Influence considers both prediction uncertainty and prediction bias. Section 4 presents empirical studies using image and text data, where we show that it requires fewer samples to achieve high predictive accuracy than existing algorithms. We conclude with a discussion of the scope and the limitations in Section 5.

2 Goal-Oriented Influence Data Acquisition

In this section, we introduce the proposed unified acquisition principle that is explicitly *goal-oriented*. Rather than optimizing a proxy (e.g., uncertainty alone), we specify a user-chosen goal objective $\mathcal{G}(\theta(\mathcal{D}))$ that encodes the scientific task of interest and depends on the model parameters θ learned from the current dataset $\mathcal{D}$. The next data point is chosen by how much it is expected to improve this goal. Directly evaluating that improvement by retraining on every candidate is infeasible, so we develop an influence-based approximation: using classical influence functions, we estimate the first-order change of $\mathcal{G}$ induced by (up)weighting a candidate, yielding a tractable criterion that couples $\nabla_\theta \mathcal{G}$, the (inverse) Hessian of the training loss, and the candidate’s gradient. To handle unknown outcomes and scale to modern models, we further introduce a surrogate ensemble to approximate required expectations and compute inverse–Hessian–vector products efficiently. The remainder of the section formalizes $\mathcal{G}$ for several tasks, derives the influence-based acquisition rule, and presents practical algorithms for scalable implementation.

2.1 Goal Objective Function

For a scientific task that can be formulated as an optimization problem, we consider the goal objective $\mathcal{G}$ as a function of the model parameters $\theta := \theta(\mathcal{D})$ fitted on the collected data points $\mathcal{D}$, which represents the most up-to-date belief. The goal objective function can be flexibly defined with respect to different tasks. For a global optimization problem, $x_{\min} = \arg \min_x f(x)$, of some unknown function f , where one could only observe values $y = f(x) + \epsilon$ of any given x with noise ϵ , such as the material design example and the hyperparameter tuning example, the goal objective function $\mathcal{G}$ is defined as the expected property value of the minimizer $\hat{x}_{\min}(\theta)$,

$$\mathcal{G}^{\text{opt}}(\theta) := \mathbb{E}_{y \sim p_0(\cdot|x)}[y \mid \hat{x}_{\min}(\theta)]. \quad (4)$$

where $\hat{x}_{\min}(\theta)$ is by minimizing the surrogate model with parameters θ ,

$$\hat{x}_{\min}(\theta) = \arg \min_x \mathbb{E}_{y \sim p_\theta(\cdot|x)}[y \mid x]. \quad (5)$$

For a learning problem where the goal is to optimize the model classification performance on some (unlabeled) target dataset $\mathcal{U}$ that shares the same conditional distribution $p_0(y \mid x)$, the goal objective function $\mathcal{G}$ is then set to be the target utility function that measures the model classification performance. For example, if the task cares about the negative log-likelihood on $\mathcal{U}$,

$$\mathcal{G}^{\text{nl}}(\theta) := - \sum_{x \in \mathcal{U}} \mathbb{E}_{y \sim p_0(\cdot|x)}[\log p_\theta(y \mid x)]. \quad (6)$$

If the task aims at maximizing the model performance on hard examples in $\mathcal{U}$ by evaluating focal loss,

$$\mathcal{G}^{\text{foc}}(\theta) := - \sum_{x \in \mathcal{U}} \mathbb{E}_{y \sim p_0(\cdot|x)}[(1 - p_\theta(y \mid x))^\gamma \log p_\theta(y \mid x)], \quad (7)$$

where γ is the pre-defined focusing parameter or the relaxation parameter.

If instead the goal is to minimize the negative model uncertainty on $\mathcal{U}$, measured by the Shannon entropy of the predicted distribution p_θ , then the corresponding goal objective function is

$$\mathcal{G}^{\text{ent}}(\theta) := - \sum_{x \in \mathcal{U}} \mathbb{E}_{y \sim p_\theta(\cdot|x)} [\log p_\theta(y|x)]. \quad (8)$$

We note that $\mathcal{G}^{\text{ent}}(\theta)$ and $\mathcal{G}^{\text{nl}}(\theta)$ measure the same utility function but under different expectations, the former one is evaluated with respect to model-posterior, while the latter is to the true conditional distribution.

Overall, the goal objective functions can be adapted to a wide range of tasks and problems. Without loss of generality, we focus on tasks or problems that can be formulated in the minimization of the goal objective function.

2.2 Influence Function

2.2.1 Maximizing informativeness

Given the goal objective function $\mathcal{G}$, we select the most “informative” candidate training data point x by evaluating how much x would affect the model update, and therefore the reduction of value in $\mathcal{G}$.

Formally, a data point x is informative if it can largely reduce the value of $\mathcal{G}$ after being acquired. The informativeness of any candidate training data point x_c given the acquired training data set $\mathcal{D}$ is measured by the reduction in the goal objective function value. See Definition 1.

Definition 1 (Informativeness) *Given an objective function $\mathcal{G} : \mathcal{P}(\mathcal{X}) \rightarrow \mathbb{R}$ defined on a set of values from the space input $\mathcal{X}$, the informativeness of an input data point $x_c \in \mathcal{X}$ can be measured by the expected change in $\mathcal{G}$ if x_c is added to the current dataset $\mathcal{D} \subset \mathcal{X}$*

$$\Delta\mathcal{G} := \mathbb{E}_{y_c \sim p_0(\cdot|x_c)} [\mathcal{G}(\theta(\mathcal{D} \cup \{(x_c, y_c)\}))] - \mathcal{G}(\theta(\mathcal{D})). \quad (9)$$

where $\theta(\mathcal{D})$ and $\theta(\mathcal{D} \cup \{(x_c, y_c)\})$ denote the model parameter estimates trained before and after x_c being added to $\mathcal{D}$.

For minimization problems, for x that gives a larger (absolute) reduction $|\Delta\mathcal{G}|$, equivalently, smaller $\Delta\mathcal{G}$, the more informative it is.

However, retraining the model for every possible candidate x_c even if the corresponding y_c is known to compute $\mathcal{G}(\theta(\mathcal{D} \cup \{(x_c, y_c)\}))$ is prohibitively computationally expensive, which makes it impractical to be used as the acquisition criterion. In the following, we provide an alternative solution based on the influence function [54] of x on $\mathcal{G}$, which approximates maximizing $|\Delta\mathcal{G}|$.

2.2.2 Evaluating informativeness via influence function

The influence function of x on $\mathcal{G}$ computes the instantaneous rate at which upweighting x reduces $\mathcal{G}$,

$$\mathcal{I} \circ \mathcal{G}(\theta(\mathcal{D} \cup \{(x_c, y_c)\})) := \left. \frac{\partial}{\partial \epsilon} [\mathcal{G}(\theta^\epsilon(\mathcal{D} \cup \{(x_c, y_c)\}))] \right|_{\epsilon=0}, \quad (10)$$

where $\theta^\epsilon(\mathcal{D} \cup \{(x, y)\})$ is the model parameter estimate after x is added to $\mathcal{D}$ with weight ϵ ,

$$\theta^\epsilon(\mathcal{D} \cup (x_c, y_c)) = \arg \min_{\theta} \frac{1}{|\mathcal{D}|} \sum_{(u,v) \in \mathcal{D}} \ell(\theta; u, v) + \epsilon \ell(\theta; x_c, y_c). \quad (11)$$

Optimizing influence function for maximizing informativeness In the following Proposition 1, we show that the expected change in $\mathcal{G}$ when candidate x_c is added in the training set $\mathcal{D}$ can be linearly approximated by minimizing the influence function of $\mathcal{G}$ without retraining the model.

Proposition 1 *The influence function $\mathcal{I}$ on the goal objective function $\mathcal{G}$ approximates $\Delta\mathcal{G}$, the expected change in $\mathcal{G}$.*

Proof 1 *Applying the first-order Taylor expansion on the goal objective function $\mathcal{G}(\theta^\epsilon(\mathcal{D} \cup \{(x_c, y_c)\}))$ at $\epsilon = 0$ gives*

$$\mathcal{G}(\theta^\epsilon(\mathcal{D} \cup \{(x_c, y_c)\})) = \mathcal{G}(\theta(\mathcal{D})) + \left. \left[\frac{\partial}{\partial \epsilon} \mathcal{G}(\theta^\epsilon(\mathcal{D} \cup \{(x_c, y_c)\})) \right] \right|_{\epsilon=0} \cdot \epsilon + o(\epsilon^2). \quad (12)$$

By definition of $\Delta\mathcal{G}$ in (9) and influence function $\mathcal{I} \circ \mathcal{G}$ from (10), we have

$$(\Delta\mathcal{G})(x_c | \mathcal{D}) = \mathcal{G}(\theta^\epsilon(\mathcal{D} \cup \{(x_c, y_c)\})) - \mathcal{G}(\theta(\mathcal{D})) \Big|_{\epsilon=\frac{1}{|\mathcal{D}|}} \approx \frac{1}{|\mathcal{D}|} \mathcal{I} \circ \mathcal{G}(\theta(\mathcal{D} \cup \{(x_c, y_c)\})). \quad (13)$$

Therefore, we can linearly approximate the expected change in test loss $\Delta\mathcal{G}$ due to adding x_c without retraining the model by computing $\frac{1}{|\mathcal{D}|} \mathcal{I} \circ \mathcal{G}$.

Recall that the most informative data point is the one that gives the largest (absolute) expected change in the goal objective function, $|\Delta\mathcal{G}|$, which is equivalent to the one that minimizes $\Delta\mathcal{G}$ under a minimization context. Proposition 1 provides a way to linearly approximate $\Delta\mathcal{G}$ by computing the influence function of $\mathcal{G}$; therefore, selecting the most informative data point is equivalent to seeking the minimizer of the influence function values, the most negatively influential data point (i.e. “helpful”) that gives the largest (absolute) expected reduction in the goal objective function $|\Delta\mathcal{G}|$ [65]. This is the precise sense in which the criterion is “good”: it is the greedy choice for reducing $\mathcal{G}$ under a first-order influence approximation.

Derivation of an explicit influence function The explicit form of the influence function on $\mathcal{G}$ in (10) can be derived by first applying the chain rule:

$$\mathcal{I} \circ \mathcal{G}(\theta(\mathcal{D} \cup \{(x_c, y_c)\})) = \left[\nabla_{\theta} \mathcal{G}(\theta(\mathcal{D})) \right]^{\top} \cdot \left[\frac{\partial}{\partial \epsilon} \theta^{\epsilon}(\mathcal{D} \cup \{(x_c, y_c)\}) \right] \Big|_{\epsilon=0}, \quad (14)$$

then a classical Taylor expansion on the partial $\frac{\partial}{\partial \epsilon} \theta^{\epsilon}(\mathcal{D} \cup \{(x_c, y_c)\})$ in (14) at $\epsilon = 0$, using the first-order optimality condition of $\theta^{\epsilon}(\mathcal{D} \cup \{(x_c, y_c)\}) \rightarrow \theta(\mathcal{D})$ as $\epsilon \rightarrow 0$ [25],

$$\left. \frac{\partial}{\partial \epsilon} \theta^{\epsilon}(\mathcal{D} \cup \{(x_c, y_c)\}) \right|_{\epsilon=0} \approx -H_{\theta}^{-1} \left[\nabla_{\theta} \ell(\theta; x_c, y_c) \right] \Big|_{\theta=\theta(\mathcal{D})}, \quad (15)$$

where H_{θ} is the Hessian matrix

$$H_{\theta} := \frac{1}{|\mathcal{D}|} \sum_{(u,v) \in \mathcal{D}} \nabla_{\theta}^2 \ell(\theta; u, v). \quad (16)$$

Combining (14) and (15) gives a closed-form expression for approximating the $\mathcal{I} \circ \mathcal{G}(\theta(\mathcal{D} \cup \{(x_c, y_c)\}))$, applied with expectation over y_c ,

$$\mathbb{E}_{y_c \sim p_0(\cdot|x_c)} [\tilde{\mathcal{I}} \circ \mathcal{G}(\theta(\mathcal{D} \cup \{(x_c, y_c)\}))] = - \left[\nabla_{\theta} \mathcal{G}(\theta(\mathcal{D})) \right]^{\top} H_{\theta}^{-1} \left[\nabla_{\theta} \mathbb{E}_{y_c \sim p_0(\cdot|x_c)} \ell(\theta; x_c, y_c) \right] \Big|_{\theta=\theta(\mathcal{D})}. \quad (17)$$

Hereby, we conclude the derivation of the explicit approximation form of the influence function of the goal objective function $\mathcal{G}$, without access to $\theta(\mathcal{D} \cup \{(x_c, y_c)\})$. This approximation enables us to obtain the most informative training data point for $\mathcal{G}$ by acquiring x_c that minimizes (17), without retraining the model for every candidate training data.

2.3 A Practical and Scalable Implementation and Algorithms

By definition, the influence function allows us to trace the optimization of $\mathcal{G}$ through the learning of θ and back to its training data, thereby identifying the training data point that has the strongest potential to induce a reduction in $\mathcal{G}$. Proposition 1 further states that minimizing the influence function gives an efficient approximation to maximizing $|\Delta\mathcal{G}|$. However, the computation of the influence function in (17) is not straightforward. In particular, the influence function computation of x_c on the goal objective function often involves evaluation of y under the true conditional distribution p_0 . Furthermore, as the number of acquired data points ($|\mathcal{D}|$) increases, direct computation of the inverse Hessian H_{θ}^{-1} given its definition in (16) prohibits efficient data acquisition. In this section, we present practical and scalable algorithms that efficiently compute the influence function in (17).

Approximating the unknown output variable Goal functions $\mathcal{G}(\theta)$ are often evaluated on unknown target outputs, for example, (4) for iterative global optimization and (6) for active learning. At the same time, $\tilde{\mathcal{I}} \circ \mathcal{G}$ is computed on $\theta(\mathcal{D} \cup \{(x_c, y_c)\})$ with y_c , which remains unknown before any acquisition takes place. The computation of the influence function for any candidate data point x_c requires some form of approximation of the corresponding y_c . To our knowledge, existing works that use the influence function for iterative data acquisition are under an active learning setting [127, 119]. A recent work chooses goal objective functions such as negative prediction entropy (8) and modified Fisher Information Ratio [130], both evaluated purely based on current training information p_{θ} and do not directly aim to improve the test performance [127]. Another line of work improves the test performance by designing the acquisition function through studying the upper-bound of test loss, which boils down to selection based on gradient norm of the loss on the candidate $\|\nabla_{\theta} \mathbb{E}_{y_c \sim p_{\theta}(\cdot|x_c)} [\ell(\theta; x_c, y_c)]\|$, and avoids any direct evaluation of the test performance [119]. When it comes to terms that involve the unknown candidate output y_c , they both approximate it with the current estimated model p_{θ} purely based on training information. In this work, we target the goal objective function (test loss under the active learning setup, or optimized minimal value under the iterative global optimization setup) under the unknown true data generating distribution p_0 , and we directly approximate the unknown output without resorting to posterior based on p_{θ} .

We consider a second surrogate model $\tilde{\mathcal{M}}_\phi$ with model parameter ϕ to be an ensemble of r neural networks, each with a different model parameter initialization on different subsets of the training data. This idea is closely related to deep ensembles, which have been empirically shown to be promising to improve accuracy, uncertainty, and out-of-distribution robustness of deep learning models [31]. In particular, we rely on the Jackknife resampling technique [116] to enhance the predictive stability and reliability. In practice, each of the r models is trained on a Jackknife subsample drawn from the available training data points, and the ensemble returns the average outputs from each model. The Jackknife estimation step has computational complexity $O(rm)$ with m parameters.

Computing the inverse Hessian vector product The evaluation of $\tilde{\mathcal{I}} \circ \mathcal{G}$ in (17), the influence function of x_c on $\mathcal{G}$ involves the inverse of the Hessian defined in (16). Assume model $\mathcal{M}$ contains m trainable parameters. Given the current acquired dataset $|\mathcal{D}|$ with total n training data points, the computation of the inverse Hessian requires $O(nm^2 + m^3)$ operations. This is particularly computationally expensive for large neural network models, which may have millions of parameters.

To avoid explicitly computing and storing the inverse Hessian, we approximate the implicit inverse Hessian-vector products (HVPs), which only requires the evaluation of HVPs in linear time with respect to the number of parameters and has been efficiently implemented in the autograd systems such as in PyTorch and JAX. For example, one can use conjugate gradient (CG) algorithms [79]. It solves an optimization problem $\min_u u^\top (H^2 + \lambda I) u - (Hv)^\top u$, the solution u^* to which approximates the Hessian inverse product $H^{-1}v$. With a solution that takes L iterations, it takes $O(nL)$ time. In practice, the optimization can be further sped up with a few iterations [78]. In the scenario where the size of the training data sets becomes very large, the CG method can still be slow, as the computation time is linear in n for going through all available training data points in each iteration. Alternatively, one can use the LiSSA algorithm developed by [2], which recursively computes the inverse HVP estimation $\hat{H}_j^{-1}v = v + (I - H) \hat{H}_{j-1}^{-1}v$ initialized with $\hat{H}_0^{-1}v = v$. It stochastically approximates the inverse HVPs with fewer training data points involved at each iteration. With a batch size B and L iterations, the approximation has computational complexity $O(nm + BLm)$. Both these algorithms are useful for solving large-scale problems characterized by low memory requirements. Empirically, we found that these two methods do not make a significant difference in terms of overall acquisition performance.

A general efficient algorithm In practice we acquire

$$x_{\text{next}} = \arg \min_{x_c} \mathbb{E}_{y_c \sim p_\phi(\cdot|x_c)} [\tilde{\mathcal{I}} \circ \mathcal{G}_{\tilde{\mathcal{M}}_\phi}(\theta(\mathcal{D} \cup \{(x_c, y_c)\}))], \quad (18)$$

with its explicit form

$$\tilde{\mathcal{I}} \circ \mathcal{G}_{\tilde{\mathcal{M}}_\phi}(\theta(\mathcal{D} \cup \{(x_c, \hat{y}_c)\})) = -\left[\nabla_\theta \mathcal{G}_{\tilde{\mathcal{M}}_\phi}(\theta(\mathcal{D}))\right]^\top \hat{H}_\theta^{-1} \left[\nabla_\theta \ell(\theta; x_c, y_c)\right] \Big|_{\theta=\theta(\mathcal{D})}, \quad (19)$$

where $\hat{\mathcal{G}}$ and $\hat{y}_c$ denote the goal objective function and candidate output evaluated approximated with the surrogate model $\tilde{\mathcal{M}}$ without direct access to p_0 , respectively; and $\hat{H}_\theta^{-1}$ denotes the stochastic approximation via implicit HVPs.

We present the general algorithm in Algorithm 2.

Algorithm 2 Goal-Oriented Influence iterative data acquisition

Input: The initial data set $\mathcal{D}$

- 1: **repeat**
 - 2: Update both main model $\mathcal{M}_\theta$ and the ensemble model $\tilde{\mathcal{M}}_\phi$ on $\mathcal{D}$
 - 3: Select $x_{\text{next}} \leftarrow \arg \min_x \mathbb{E}_{y \sim p(\cdot|x)} [\tilde{\mathcal{I}} \circ \mathcal{G}_{\tilde{\mathcal{M}}_\phi}(\theta(\mathcal{D} \cup \{(x, y)\}))]$ {see closed-form approximation in (19)}
 - 4: Query $y_{\text{next}} \leftarrow \text{OBSERVE}(x_{\text{next}})$
 - 5: Augment $\mathcal{D} \leftarrow \mathcal{D} \cup \{(x_{\text{next}}, y_{\text{next}})\}$
 - 6: **until** termination condition is met {e.g. budget exhausted or desired goal achieved}
-

3 Goal-Oriented Influence Data Acquisition in Exponential families

In this section, we derive Goal-Oriented Influence data acquisition for exponential family distributions. We first formalize the exponential-family setup and derive the Goal-Oriented Influence in closed form. In Section 3.1, we give an interpretation of the influence function geometrically, explain the role of the bias term, and compare Goal-Oriented Influence with predictive-entropy acquisition—positioning Goal-Oriented Influence as an *exploration-aware, bias-focused improvement surrogate* that does not require a posterior over θ . Finally, we instantiate the framework for three

use cases in section 3.2: *deep active learning*, where the goal is to learn a classification model that does well on test data with fewest acquired labeled samples, *global optimization on noisy functions*, where the goal is to find the minimizer for the complex expensive-to-evaluate noisy function, *hyperparameter tuning*, where the goal is to minimize the number of sets of hyperparameters used to train and validate the model, while achieving the best performance on the test data (test loss).

Model setup Formally, consider $x \in \mathbb{R}^d$, $y \in \mathbb{R}$, we model $y \sim p_\theta(\cdot | x)$ with an exponential family of probability distributions in the following general form

$$p_\theta(y | x) = h(y) \exp \left(\eta_\theta(x) T(y) - A(\eta_\theta(x)) \right), \quad (20)$$

where $T(y)$ is the sufficient statistic and $\eta_\theta(x)$ is the natural parameter with model parameters θ ¹. Exponential families have many appealing statistical and computational properties,

(3.1) $A(\eta)$ is a convex function;

(3.2) $\mathbb{E}_\eta[T(y)] = A'(\eta)$;

(3.3) $\text{var}_\eta[T(y)] = A''(\eta)$.

Under this context, η is modeled by a deep neural network with parameters θ , trained with the negative log-likelihood loss $\ell(\theta; x, y) := -\log p_\theta(y | x)$, explicitly,

$$\ell(\theta; x, y) = -\log h(y) - \eta_\theta(x) T(y) + A(\eta_\theta(x)). \quad (21)$$

Influence function By definition of $\ell(\theta; x, y)$ in (21), we have

$$\nabla_\theta \mathbb{E}_{y_c \sim p_0(\cdot | x_c)} [\ell(\theta; x_c, y_c)] = \nabla_\theta \eta_\theta(x_c) [A'(\eta_\theta(x_c)) - A'(\eta_0(x_c))],$$

substituting which into (17) gives

$$\text{GOI}(x_c) = -[\nabla_\theta \mathcal{G}(\theta)]^\top H_\theta^{-1} \nabla_\theta \eta_\theta(x_c) [A'(\eta_\theta(x_c)) - A'(\eta_0(x_c))]. \quad (22)$$

When $T(y) = y$, the term $A'(\eta_\theta(x_c)) - A'(\eta_0(x_c))$ in (22) equals $\mathbb{E}_\theta[y | x_c] - \mathbb{E}_0[y | x_c]$, that is, the *prediction bias*. That is, the optimal data point to acquire is $x_{\text{next}} = \arg \min_{x_c} \text{GOI}(x_c)$ from (22).

3.1 Understanding Influence Functions under Exponential Families Assumption

Geometric interpretation of the influence function The resulting acquisition criterion (22) selects the point whose (curvature-aware) parameter push best aligns with the direction that most reduces the goal, and does so preferentially where the model is currently biased. Let $b(x_c) := A'(\eta_\theta(x_c)) - A'(\eta_0(x_c))$ and $\mu := H_\theta^{-1/2} \nabla_\theta \mathcal{G}(\theta)$, $\nu(x_c) := H_\theta^{-1/2} \nabla_\theta \eta_\theta(x_c)$, we then have

$$\text{GOI}(x_c) = - \underbrace{\langle \mu, \nu(x_c) \rangle}_{\substack{\text{goal alignment} \\ \text{in the } H\text{-geometry}}} \times \underbrace{b(x_c)}_{\substack{\text{prediction bias} \\ \mathbb{E}_\theta[y | x_c] - \mathbb{E}_0[y | x_c]}}. \quad (23)$$

1. *Direction of improvement* ($\nabla_\theta \mathcal{G}(\theta)$). The inner product uses the gradient of the *actual goal* as a sensitivity vector—so we prioritize candidates whose parameter influence most reduces the quantity we care about (test loss, task utility, etc.).
2. *Curvature adaptivity* (H_θ^{-1}). Preconditioning by H_θ^{-1} turns the Euclidean gradient into a *natural/second-order* step: directions with high curvature (well-determined parameters) are down-weighted; uncertain, flat directions are amplified. This yields approximate reparameterization-invariance and targets the directions where additional data can move the model.
3. *Leverage of the candidate* ($\nabla_\theta \eta_\theta(x_c)$). This is the candidate’s *score direction*: how adding (x_c, y_c) tends to perturb the parameters. Its norm in the H^{-1} -metric is a generalized leverage score.
4. *Bias targeting* ($b(x_c)$). If the model is already calibrated at x_c , then $b(x_c) \approx 0$ and the expected first-order improvement vanishes—there is little to gain from labeling this point. Large $|b(x_c)|$ steers acquisition to regions where the model’s predictive mean disagrees with p_0 , converting leverage into actual goal improvement.

¹Notation wise, we do not explicitly differentiate between scalar and vector parameters.

The bias term Applying a first-order Taylor expansion on $A'(\eta_0(x_c))$ at $\theta_0 = \theta$ yields

$$b(x_c) := A'(\eta_\theta(x_c)) - A'(\eta_0(x_c)) \approx \nabla_\theta \eta_\theta(x_c)^\top (\theta - \theta_0) A''(\eta_\theta(x_c)). \quad (24)$$

Substituting into (23) gives

$$\text{GOI}(x_c) = - \underbrace{\langle \mu, \nu(x_c) \rangle}_{\text{goal alignment in the } H\text{-geometry}} \times \underbrace{\nabla_\theta \eta_\theta(x_c)^\top (\theta - \theta_0)}_{\text{directional parameter bias at } x_c} A''(\eta_\theta(x_c)). \quad (25)$$

which makes the dependence on the *parameter bias* $\theta - \theta_0$ explicit.

While the geometric form (23) highlights *prediction bias* at the output level, the approximation in (25) replaces the unobservable term $A'(\eta_0)$ by the *parameter bias* $(\theta - \theta_0)$. This attributes bias to concrete parameter directions through $\nabla_\theta \eta_\theta(x)$ and recovers familiar GLM structure ($b(x) \approx A''(\eta_\theta(x)) x^\top (\theta - \theta_0)$), which we leverage in the following discussion on prediction entropy comparison.

Connection to predictive entropy (PE) minimization While the most widely used acquisition criterion is uncertainty-based, influence function-based acquisition do not explicitly rely on uncertainty estimates of model $\mathcal{M}_\theta$. When $\mathcal{M}_\theta$ is a neural network, obtaining uncertainty estimates is often computationally expensive to obtain and unreliable due to the small size of initial training dataset. On the other hand, there is a close connection between influence function minimization and predictive entropy minimization in Bayesian generalized linear models, stated in Proposition 2.

Proposition 2 *Influence function-based minimization approximates predictive entropy minimization in generalized linear models (GLMs) with a canonical link. Under mild assumptions, the two objective functions differ by an additional term that calibrates the directional parameter bias up to a constant.*

Proof 2 Consider a generalized linear model in the form of (20) with a canonical link $\eta_\theta(x) = x^\top \theta$ and $T(y) = y$. The conditional entropy of model after acquiring (x_c, y_c) can be derived as

$$\mathcal{H}(\theta \mid \mathcal{D} \cup \{(x_c, y_c)\}) = \frac{1}{2} \mathbb{E}_{y_c | x_c} [\log |H_{\theta_c}^{-1}|] + \text{const} \quad (26)$$

where H_{θ_c} is the Hessian of negative log likelihood with model parameter $\theta_c = \theta(\mathcal{D} \cup \{(x_c, y_c)\})$.

Under the canonical-link assumption and properties (3.2) and (3.3), minimizing (26) can be simplified to minimize [70]

$$\mathbb{E}_{y_c | x_c} [\log |H_{\theta_c}^{-1}|] = \log |H_\theta^{-1}| - \mathbb{E}_{y_c | x_c} \log \left(1 + x_c^\top H_\theta^{-1} x_c A''(\eta_\theta(x_c)) \right). \quad (27)$$

Given that $A''(\eta_\theta(x_c))$ is bounded and $x_c^\top H_\theta^{-1} x_c \rightarrow 0$ as $|\mathcal{D}| \rightarrow \infty$ [84], when $z = x_c^\top H_\theta^{-1} x_c A''(\eta_\theta(x_c))$ is sufficiently small, the standard linear approximation $\log(1 + z) = z + o(z)$ further simplifies the acquisition objective function as

$$\text{PE}(x_c) := -x_c^\top H_\theta^{-1} x_c A''(\eta_\theta(x_c)) \quad (28)$$

On the other hand, the acquisition score under Goal-Oriented Influence (25) is

$$\text{GOI}(x_c) := -[\nabla_\theta \mathcal{G}(\theta)]^\top H_\theta^{-1} x_c \underbrace{x_c^\top (\theta - \theta_0)}_{\text{prediction bias}} A''(\eta_\theta(x_c)), \quad (29)$$

where the additional directional parameter bias term coincides with the prediction bias under the context GLMs with a canonical link.

By comparing objective in (28) with the one in (29), Goal-Oriented Influence differs by multiplicative goal-alignment and bias weights:

$$\text{GOI}(x_c) = \text{PE}(x_c) \times \frac{\langle H_\theta^{-1/2} \nabla_\theta \mathcal{G}(\theta), H_\theta^{-1/2} x_c \rangle}{\|H_\theta^{-1/2} x_c\|^2} \times [x_c^\top (\theta - \theta_0)]. \quad (30)$$

This observation suggests that Goal-Oriented Influence is implicitly calibrating model uncertainty. The only difference is that Goal-Oriented Influence weighs each dimension of the parameter by its current bias, whereas predictive entropy minimization weighs all the dimensions equally.

Proposition 2 shows that in canonical GLMs, Goal-Oriented Influence shares the same H_θ^{-1} leverage structure $A''(\eta_\theta(x_c)) x_c^\top H_\theta^{-1} x_c$ as predictive-entropy selection, and calibrates it by (i) a *goal-alignment* term that projects onto the descent direction of $\mathcal{G}$, and (ii) a *prediction-bias* term that emphasizes inputs where the model disagrees with the data-generating mechanism. Thus, Goal-Oriented Influence is *uncertainty-aware*—it retains the PE signal—yet it *calibrates* that signal toward directions that most reduce the stated goal and toward regions of systematic error, without requiring a posterior over θ .

Consequently, although Goal-Oriented Influence is derived as a goal-directed influence minimizer (exploitation), it is *uncertainty-aware* in the sense that it preferentially scores candidates in high-leverage, high-variance regions—thus *implicitly encouraging exploration*. In active learning, though prediction calibration (bias) is rarely considered, it emphasizes mislabeled or misfit regions, and the empirical simulations (see Section 4.1) suggest it achieves target accuracy with substantially fewer labels than dropping the bias, confirming that *calibrating bias materially improves sample efficiency*. In global optimization, it plays the role of an *exploration-aware, exploitation-focused* alternative approach that mirrors the exploitation component of Bayesian optimization without requiring a posterior.

3.2 Example Use of Goal-Oriented Influence Data Acquisition

Due to the flexibility of the user-defined goal objective function, the proposed Goal-Oriented Influence data acquisition method can be widely applied to various problems. In Section 2, we introduce how to define the goal objective function to solve the global optimization problem with a complex, expensive-to-evaluate objective function that lacks an analytical form, not to mention first- or second-order derivative information, see (4). We also present the goal objective functions to be used under the deep active learning setup where the goal is to optimize the deep learning model performance on some validation or test dataset with minimum labeling costs, see (6) and (8).

In this section, we work with exponential family distributions to showcase the Goal-Oriented Influence data acquisition detailed application to solve both the global optimization and active learning problems. In each example problem, we first explicitly write out the goal objective function $\mathcal{G}$, then derive the influence-based acquisition function (19) used in Algorithm 2.

Deep active learning This section showcases how the Goal-Oriented Influence data acquisition can be used for deep active learning under the exponential family distributions assumption. We first define a test-centric goal objective function $\mathcal{G}(\theta)$, then derive a closed-form *influence score* for adding a candidate (x_c, y_c) that couples the H_θ^{-1} geometry with a *prediction-bias weight*.

Formally, consider a classification task where $y \in \mathbb{Z}$ denotes the output label and $x \in \mathbb{R}^d$ denotes the input features. This relates to the biological system response and the stimuli in the biological study motivation example, where conducting experiments to read the system response is costly. At each iteration step, a deep neural network $\mathcal{M}_\theta$ with model parameters $\theta := \theta(\mathcal{D})$ is trained on currently available dataset $\mathcal{D}$ to predict y (the response) from x (the stimuli), (x, y) is the target test data with unknown y , following the same conditional distribution p_0 . The goal is to select x_{next} to acquire its label y_{next} and append to $\mathcal{D}$ iteratively, such that we can achieve a desired low test loss $\ell(\theta; x, y)$ with minimum labeling/acquisition efforts.

To optimize the model performance on test data (x, y) where y is unknown, the goal objective function can be expressed as

$$\mathcal{G}(\theta) := \mathbb{E}_{y \sim p_0(\cdot|x)} [\ell(\theta; x, y)]. \quad (31)$$

and its gradient can be derived with the explicit form in (21)

$$\nabla_\theta \mathcal{G}(\theta) = [A'(\eta_\theta(x)) - A'(\eta_0(x))] \nabla_\theta \eta_\theta(x). \quad (32)$$

At each iteration step, the influence function in (22) can be explicitly derived as

$$\begin{aligned} \text{Influence}(x_c) &= -[A'(\eta_\theta(x)) - A'(\eta_0(x))] \nabla_\theta \eta_\theta(x)^\top H_\theta^{-1} \nabla_\theta \eta_\theta(x_c) [A'(\eta_\theta(x_c)) - A'(\eta_0(x_c))] \\ &\quad \propto -\nabla_\theta \eta_\theta(x)^\top H_\theta^{-1} \nabla_\theta \eta_\theta(x_c) [A'(\eta_\theta(x_c)) - A'(\eta_0(x_c))] \end{aligned} \quad (33)$$

Hence, the optimal data point to acquire is

$$x_{\text{next}} = \arg \min_{x_c} -\nabla_\theta \eta_\theta(x)^\top H_\theta^{-1} \nabla_\theta \eta_\theta(x_c) [A'(\eta_\theta(x_c)) - A'(\eta_0(x_c))]. \quad (34)$$

Under the canonical-link assumption with $T(y) = y$,

$$x_{\text{next}} = \arg \min_{x_c} -x^\top H_\theta^{-1} x_c \underbrace{x_c^\top (\theta - \theta_0)}_{\text{prediction bias}} A''(\eta_\theta(x_c)), \quad (35)$$

which recovers (29) with $\mathcal{G}$ defined in (31) under the active learning setup.

Practically, we estimate the bias term via a Jackknife resampling technique using the deep ensemble $\tilde{M}_\phi$ and compute inverse HVPs stochastically.

Iterative global optimization with noisy observations Consider a global optimization problem solving $x_{\min} = \arg \min_x f(x)$ of some unknown function f . In particular, one can only obtain noisy observations $y = f(x) + \varepsilon$, where ε denotes the noise variable with mean 0. At each iteration step, a model $\mathcal{M}_\theta$ with parameters $\theta = \theta(\mathcal{D})$ is trained on the acquired data $\mathcal{D} = \{(u_i, v_i)\}_{i=1}^n$ up to the current stage to approximate the true underlying function f .

We define the goal objective function $\mathcal{G}$ to be the expected value of the minimizer $\hat{x}_{\min}(\theta)$, as in (4) of (5). Under the exponential family distribution assumption with $T(y) = y$, we have

$$\mathcal{G}(\theta) := \mathbb{E}_{y \sim p_\theta(\cdot|x)}[y | x = \hat{x}_{\min}(\theta)] \stackrel{(3.2)}{=} A'(\eta_0(\hat{x}_\theta^*)) \quad (36)$$

where $\hat{x}_{\min}(\theta)$ is by minimizing the surrogate model $\mathcal{M}_\theta$ with parameters θ fitted on the up-to-date dataset $\mathcal{D}$,

$$\hat{x}_{\min}(\theta) = \arg \min_x \mathbb{E}_{y \sim p_\theta(\cdot|x)}[y | x]. \quad (37)$$

By definition of the influence function, Goal-Oriented Influence selects the candidate that maximizes the *instantaneous expected decrease* of the true objective evaluated at the surrogate’s recommended design $\hat{x}_{\min}(\theta)$. For ease of exposition, we denote $\hat{x}_\theta^* := \hat{x}_{\min}(\theta)$.

The gradient $\nabla_\theta \mathcal{G}(\theta)$ can be derived as

$$\nabla_\theta \mathcal{G}(\theta) := A''(\eta_0(\hat{x}_{\min}(\theta))) \left[\frac{\partial}{\partial \theta} \hat{x}_{\min}(\theta) \right] [\nabla_x \eta_0(\hat{x}_{\min}(\theta))]. \quad (38)$$

By the chain rule and implicit differentiation of $\hat{x}_\theta^*$ (see Appendix .. for derivation details)

$$\frac{\partial}{\partial \theta} \hat{x}_\theta^* \approx \left[\frac{\partial^2}{\partial \theta \partial x} A'(\eta_\theta(\hat{x}_\theta^*)) \right] [\nabla_x^2 A'(\eta_\theta(\hat{x}_\theta^*))]^{-1} \quad (39)$$

The Goal-Oriented Influence score in (22) can be explicitly derived as

$$\text{GOI}(x_c) = - \underbrace{A''(\eta_0(\hat{x}_\theta^*)) \nabla_x \eta_0(\hat{x}_\theta^*)^\top}_{\text{how the true objective drops if } \hat{x}_\theta^* \text{ moves}} \underbrace{\left[\nabla_x^2 A'(\eta_\theta(\hat{x}_\theta^*)) \right]^{-1} \left[\frac{\partial^2}{\partial \theta \partial x} A'(\eta_\theta(\hat{x}_\theta^*)) \right]}_{\text{how } \hat{x}_\theta^* \text{ moves when } \theta \text{ moves}} \underbrace{H_\theta^{-1} \nabla_\theta \eta_\theta(x_c) b(x_c)}_{\text{how } \theta \text{ moves when adding } (x_c, y_c)}, \quad (40)$$

where $b(x_c) := A'(\eta_\theta(x_c)) - A'(\eta_0(x_c))$ denotes the prediction bias term as in (23). Practically, we approximate $\eta_0(\cdot)$ via a Jackknife resampling technique using the deep ensemble $\tilde{M}_\phi$ and compute inverse HVPs stochastically.

Hyperparameter optimization A slightly more complex optimization problem can be formulated as hyperparameter tuning for neural networks under a transfer learning setup. These hyperparameters specify the model architecture and different sets of hyperparameters often yield different trained model parameters and therefore different test performance. It is computationally expensive to acquire the model performance on the known training dataset (training loss) for even just one set of hyperparameters. Thus the goal is to minimize the number of sets of hyperparameters used to train and validate the model, while achieving the best performance on the test data (test loss).

Formally, let ξ denote the hyperparameters of the model to be trained on the fixed training data set $\mathcal{D}^{\text{tr}} = \{(x_i^{\text{tr}}, y_i^{\text{tr}})\}_{i=1}^n$, and (x, y) denote the test data where y can be unknown². The predictive loss on test data (x, y) using the model trained with the hyperparameters ξ can be expressed as $h_0(x, \xi) := \mathbb{E}_y[\ell(\xi; x, y)]$, a function of ξ and x is given. Since models trained on the same set of hyperparameters may still yield different validation performance due to stochastic variability in model initialization, the observation of h is “noisy” in nature, that is, we can only get access to h_0 in its noisy form, $r = h_0(x, \xi) + \varepsilon$ where ε denotes some noise variable. We aim to optimize h_0 over ξ given x , as the “mean” test performance that is indifferent of stochastic variability in the model training.

Denote q as the true conditional distribution of r given the concatenated input (x, ξ) , $r \sim q_0(\cdot | x, \xi)$ with parameter θ_0 . We train a surrogate model with parameter θ on the currently acquired dataset $\mathcal{D}$, each data point of which is in the form of $([x, \xi], r)$. Under the exponential family distribution assumption,

$$q_\theta(r | x, \xi) = h(r) \exp \left(\eta_\theta(x, \xi) T(r) - A(\eta_\theta(x, \xi)) \right). \quad (41)$$

²For ease exposition, we consider only one data point

The search for the hyperparameters that yield the best test performance can be expressed with a goal objective function defined as

$$\mathcal{G}(\theta) := \mathbb{E}_{r \sim q_\theta(\cdot | x, \xi)} [r | x, \xi = \hat{\xi}_{\min}(\theta)] \quad (42)$$

where $\hat{\xi}_{\min}(\theta)$ is the minimizer of the surrogate model $\mathcal{M}_\theta$ with parameters θ fitted on the up-to-date dataset $\mathcal{D}$,

$$\hat{\xi}_{\min}(\theta) = \arg \min_{\xi} \mathbb{E}_{r \sim q_\theta(\cdot | x, \xi)} [r | x, \xi] \quad (43)$$

This problem can then be solved similarly following the iterative global optimization with noisy observations example. The only difference between $\hat{\xi}_{\min}(\theta)$ from (43) and $\hat{x}_{\min}(\theta)$ from (37) is that the optimizer $\hat{\xi}_{\min}(\theta)$ is obtained w.r.t. an additional component x . Therefore, the influence function can be derived similarly as in (40).

4 Empirical Studies

We empirically investigate GOI across a range of controlled and realistic learning and optimization tasks³. We first use a synthetic logistic-regression experiment to isolate the effect of the parameter-bias term in the acquisition rule. We then study predictive learning problems in image and text classification [69, 20, 58], comparing GOI to uncertainty-based active learning methods such as BALD[52, 38, 63]. Next, we evaluate GOI as an acquisition strategy for noisy global optimization of black-box test functions, in comparison with Gaussian-process Bayesian optimization methods [122, 98, 57, 114, 109]. Finally, we apply GOI to hyperparameter tuning on CIFAR-10 under a distribution shift between training and target data, where the goal is to select hyperparameters that improve performance on the target test distribution using only labels from a different source distribution.

4.1 Importance of the bias term

The acquisition approximation in (25) makes the dependence on the parameter bias $\theta - \theta_0$ explicit. This representation attributes bias to concrete parameter directions through $\nabla_\theta \eta_\theta(x)$, enabling diagnostics and targeted regularization, and it provides a plug-in implementation via Jackknife/Bootstrap surrogates of θ_0 using a single $H_\theta^{-1} \nabla_\theta \mathcal{G}$ solve plus per-candidate vector-Jacobian product or Jacobian-vector product. Thus the parameter-bias form is both interpretable and computationally practical.

Proposition 2 shows that, in canonical GLMs, the GOI rule shares the same H_θ^{-1} leverage structure as predictive-entropy selection; the key distinction is GOI’s bias calibration, which re-weights parameter directions by their current bias rather than treating them uniformly. Concretely, apart from the goal-alignment term, the additional factor is the prediction-bias term $x_c^\top (\theta - \theta_0)$. If we replace $\theta - \theta_0$ by a vector of ones, the objective in (29) reduces to the predictive-entropy objective (28) up to a scaling constant, recovering a bias-agnostic leverage rule.

This raises the question: how important is the bias term $\theta - \theta_0$ in practice? To isolate its effect, we conduct a simple simulation study comparing variants of the acquisition based on (29). The inputs $x \in \mathbb{R}^d$ are generated from a low-rank latent variable model $z \in \mathbb{R}^l$, $l < d$ [113]: $x = Wz + \epsilon$ with $z \sim \mathcal{N}(0, I)$ and noise $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$. We set $d = 20$, $l = 3$, and $\sigma = 0.1$. The outcome y is binary with conditional distribution $p_0(y | x)$ given by a Bernoulli GLM. We generate 50,000 training points and 5,000 test points for each of 200 repetitions. At each acquisition step, a logistic regression model is refit to the currently labeled set, and the next point is selected by minimizing (29) under different treatments of the bias term; random sampling from the pool serves as a baseline. All models start from two labeled points (one from each class), and the remaining training points form the candidate pool. Test accuracy is recorded after each acquisition.

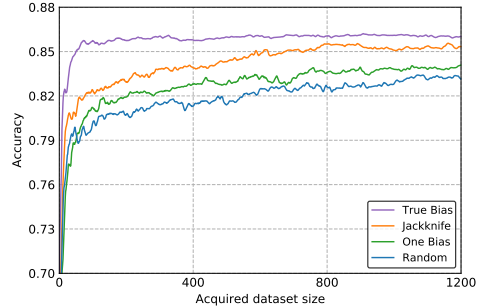


Figure 1: The bias term matters in acquisition computation. Using the acquisition function with the true bias term reaches a high accuracy rate with much fewer acquisitions. The acquisition function with a proper approximation to the true bias term beats the one dropping the bias term. Simulation test accuracy is given as a function of the number of acquired data points from the pool set (averaged over 200 repetitions).

³The codes to reproduce the results of our empirical studies are found on GitHub repository, github.com/pamplermousse1011/Influence.

Figure 1 compares four acquisition strategies:

1. *True Bias* – minimizing the influence function with the true bias term $\theta - \theta_0$;
2. *Jackknife* – minimizing the influence function with a Jackknife estimate of $\theta - \theta_0$;
3. *One Bias* – minimizing the influence function with $\theta - \theta_0$ replaced by a vector of ones (i.e., ignoring bias calibration);
4. *Random* – random acquisition from the pool.

As expected, *True Bias* attains high accuracy with the fewest acquisitions, and *Jackknife* closely tracks its performance. While *One Bias* still outperforms random acquisition, it consistently lags behind the bias-aware variants. Ignoring the bias term therefore hurts overall acquisition efficiency: the better we approximate $\theta - \theta_0$, the faster the acquisition reaches high accuracy and the fewer labeled points are required.

4.2 Predictive learning

In our experiments, we repeatedly go through active learning loops where we first train the model on the acquired data points and then select the new data point from a given pool of data S based on the different acquisition methods. Throughout the experiments, all methods are assessed with a similar model structure: feedforward neural networks composed of multiple `linear-relu` layers, ending with a `softmax` layer. Dropout layers are added in BALD to perform approximate inference [38]. During the training process, we use Adam optimizer [62] with a learning rate 0.001 and momentum parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$. We train both BALD and GOI with one GPU. The test accuracy of each model was assessed after each acquisition. All figures and tables are constructed from four independent trials, with error bars indicating the mean together with the lower and upper quartiles.

MNIST The MNIST database contains 60,000 training images and 10,000 test images of handwritten digits. Similar to [38], all models are trained with a random but balanced initial training set of 20 data points (two samples per class), and a validation set of 1024 points is set aside to optimize the weight decay. The rest of the data points are used as a pool set. GOI and Random acquisition are assessed with the same model structure: three layers of `linear-relu`, with hidden unit dimensions 392 and 128; BALD adopts a similar structure but with two dropout layers for inference: two layers of `linear-dropout-relu` followed by the final output layer `linear-softmax`, with the same hidden unit dimensions. We use 100 MC dropout samples in BALD. To compute the expected loss for the test data points and the expected loss for candidate data points, we use 10 Jackknife samples in an approximation of the expected prediction loss.

The results are presented in Table 1 and Figure 2. Table 1 gives the number of acquisition steps needed to achieve the accuracy rate of 80%, 90%, and 95%. For the MNIST dataset, GOI can achieve a high accuracy rate with much fewer acquisitions than BALD and Random. With GOI, an expert is able to label less than a quarter of the number of images compared to random sampling. Table 1 highlights the data efficiency and the importance of active learning. The data in Figure 2 shows the accuracy rate at each iteration, demonstrating that GOI outperforms both random acquisition and BALD.

EMNIST The EMNIST Letters dataset merges a balanced set of uppercase and lowercase letters and provides a single 26-class task. It contains 124800 training images and 20800 test images. The last portion of the training set, equal in size to the test set, is set aside as a validation set. The remaining data points are used as a pool set. Similar to [38], we perform an initial acquisition step with a randomly initialized model.

Similar to what is used in MNIST, GOI and Random have three `linear-relu` layers with hidden unit dimensions 392 and 128, respectively, and BALD adds two additional dropout layers for inference. In addition, 50 MC dropout samples are used in BALD for acquisition. To compute the expected loss for the test data points and the expected loss for candidate data points, 5 Jackknife samples are used to approximate the expected loss.

The results for the acquisition of up to 250 data points are presented in Figure 2. GOI outperforms both random acquisition and BALD. We observe it gives around 2% higher accuracy compared to BALD and random acquisition. For this dataset, BALD gives only slightly better performance than random acquisition.

Table 1: **GOI requires fewer samples than BALD and random acquisition to achieve high accuracy on MNIST.** The table reports 25%-, 50%-, and 75%-percentiles for the number of required data points to reach 80%, 90%, and 95% accuracy on MNIST.

% Accuracy	Acquisition Methods		
	GOI	BALD	Random
80%	88/108/116	104/109/117	138/150/173
90%	374/422/441	432/445/475	846/901/977
95%	1060/1150/1206	1514/1589/1678	4265/4522/4731

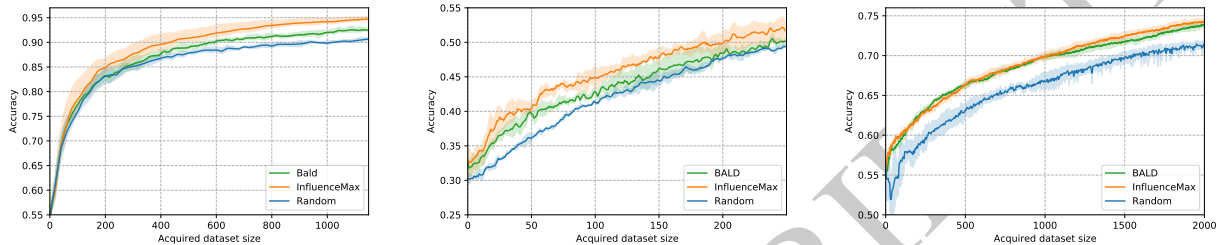


Figure 2: **GOI outperforms both random acquisition and BALD on classification tasks on images of digits from MNIST (Left) and EMNIST (Middle) and the sentiment of movie reviews from the Rotten Tomatoes dataset (Right).** The test accuracy is evaluated at each acquisition step. Left: GOI outperforms both random acquisition and BALD on MNIST. Middle: GOI outperforms both random acquisition and BALD on EMNIST, whereas BALD only performs slightly better than random acquisition. Right: Both GOI and BALD outperform the random sampling in terms of the accuracy rate at a given acquired dataset size. While GOI and BALD have similar performance, GOI gives slightly higher accuracy rates after the total acquisition size reaches 1,000.

Movie reviews from Rotten Tomatoes The Rotten Tomatoes movie review dataset is a corpus of movie reviews used for sentiment analysis, originally collected by Pang and Lee [83]. In the original data set, phrases are labeled on a scale of five values: negative, somewhat negative, neutral, somewhat positive, and positive. To adapt the dataset for a binary classification task, we removed the phrases labeled with neutral, relabeled “negative” and “somewhat negative” with 0, and “somewhat positive” and “positive” with 1. We randomly split the total 76478 phrases into a training set of size 59070, a validation set of size 1024, and a test set of size 16384. The initial training set contains a balanced dataset of size 20. Before applying the neural network models, we use the Bag-of-Words model (BoW) to extract features from the text. This process converts text into a matrix of the occurrence of words from the reviews. We also discard words with occurrence less than 10, which reduces the total number of features from 15186 to 7004. GOI and Random have three linear-relu layers with hidden unit dimensions 64 and 64, respectively. BALD again adds two additional dropout layers for inference. In addition, 50 MC dropout samples are used in BALD for acquisition. To compute the expected loss for the test data points and the expected loss for candidate data points, 10 Jackknife samples are used in approximation of the expected loss.

Figure 2 displays the performance comparison of the three acquisition methods. GOI and BALD outperform the random acquisition at each acquisition step with accuracy rates around 4% higher. The width of the error bars also indicates that GOI and BALD can provide more stable results compared to sampling at random.

4.3 Noisy function optimization

We evaluate the methods for optimization of synthetic test functions chosen from [12]. In particular, we are interested in the noisy setup, i.e., at each acquisition, one can only get access to the noisy observation $y = f(x) + \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. In our experiments, we set $\sigma = 0.1$ and $\sigma = 0.2$. In all cases, 5 initial noisy observations of the function f are given. Figure 3 provides a performance comparison on 2D Branin and 5D Ackley functions between GOI and five other commonly used Gaussian Processes (GP)-based Bayesian optimization methods with different acquisition functions, namely, probability of improvement (PI) [114], expected improvement (EI) [57], upper confidence bound (UCB) [109], max-value entropy search (MES) [122], and knowledge gradient (KG) [98, 33, 34]. In each of these

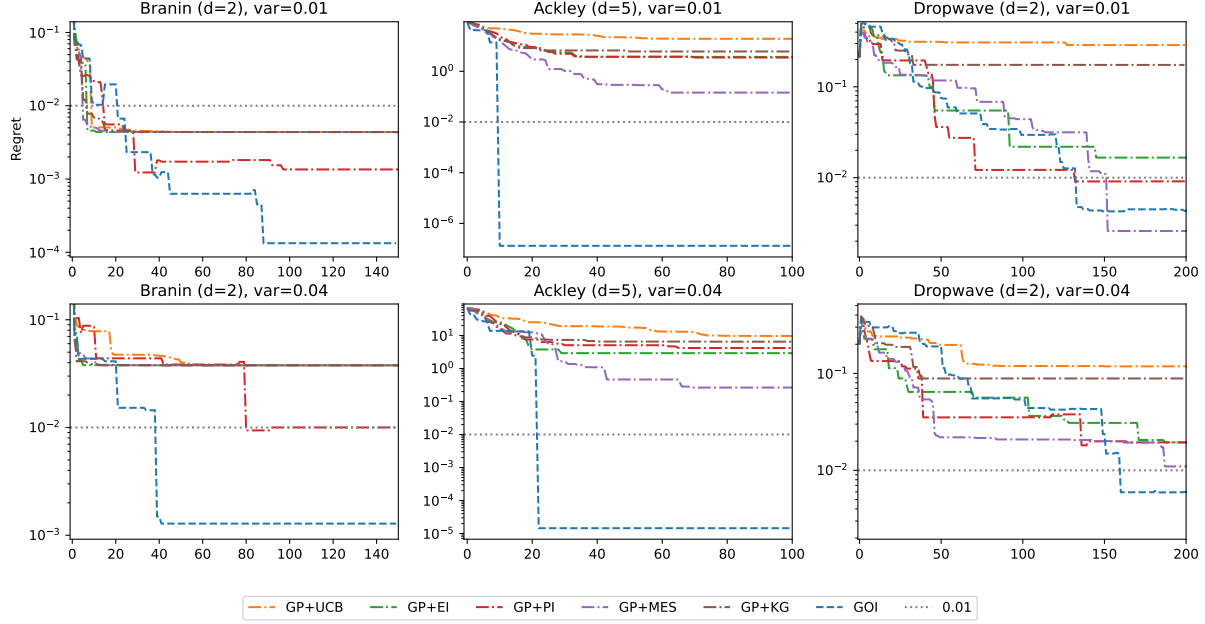


Figure 3: **GOI finds the minimum of the noisy objective functions with the least number of acquisition steps compared to Bayesian optimization methods.** Immediate regret is evaluated at each acquisition step on learning noisy Branin, Ackley and Dropwave functions. As the noise level goes up, the performance of GOI can further improve over the other Bayesian optimization methods.

plots, we show the *immediate regret*, which measures the difference between the outcome of the best possible decision ($\min_{x \in S} f(x)$ for some feasible set S) and the outcome of the decision made by each active optimization method. Given its noisy nature, the regret value may go up in consecutive acquisition steps but should in general decline if solved by an effective optimization algorithm.

2D Branin The first function is the Branin function $f(\mathbf{x}) = a[(x_2 - bx_1^2 + cx_1 - r)^2 + s(1 - t)\cos(x_1) - q]$ where $a = 1/51.95$, $b = 5.1/(4\pi)^2$, $c = 5/\pi$, $r = 6$, $s = 10$, $t = 1/(8\pi)$ and $q = 44.81$, a rescaled form by [86]. The function is evaluated on the square $x_1 \in [0., 1.]$, $x_2 \in [0., 1.]$. It has three global minima $f(\mathbf{x}^*) = -1.0474$.

2D Drop-Wave function This two-dimensional, radially symmetric test function is highly multimodal due to its oscillatory cosine term. For $\mathbf{x} = (x_1, x_2)$ it is defined as

$$f(\mathbf{x}) = -\frac{1 + \cos(12\sqrt{x_1^2 + x_2^2})}{0.5(x_1^2 + x_2^2) + 2},$$

and is typically evaluated on $x_i \in [-5.12, 5.12]$. It attains the global minimum $f(\mathbf{x}^*) = -1$ at $\mathbf{x}^* = (0, 0)$.

5D Ackley function The Ackley function is generally expressed in the form $f(x) = -a \exp(-b\sqrt{\frac{1}{d} \sum_i x_i^2}) - \exp(\frac{1}{d} \sum_i \cos(cx_i)) + a + \exp(1)$, where $a = 20$, $b = 0.2$, $c = 2\pi$ and $\sigma = 0.5$. For the *dimension* $d = 2$, the function leads some optimization algorithms, particularly hill-climbing algorithms, to be trapped in one of its many local minima. The function is evaluated on the hypercube $x_i \in [-5, 5]$, for $i = 1, \dots, d$. We set the dimension $d = 5$ for higher dimensional analysis. The *Global minimum* $f(\mathbf{x}^*) = 0$ is achieved at $\mathbf{x}^* = (0, \dots, 0)$.

4.4 Hyperparameter tuning

We here consider a hyperparameter optimization on some target data set $(\tilde{X}, \tilde{Y})$ with unknown label $\tilde{Y}$ when the model can only be trained on a data set (X, Y) , where $X, \tilde{X} \sim \Omega$ and $Y, \tilde{Y} \in \mathcal{Y}$. Note that the underlying distribution p_X can be very different from $p_{\tilde{X}}$ but share the same conditional distribution of the form $p_0 := p(Y|X; \theta_0)$ with some unknown parameters θ_0 . Consider the CIFAR10 database [66] that contains 60000 training images in 10

classes $\mathcal{Y} = \{C_1, C_2, \dots, C_{10}\}$ and 10000 test images. The model of interest is a Pre-Activation Residual Neural Network [49]. In this example, the target data set $(\tilde{X}, \tilde{Y})$ is composed of the 3000 test images, whose label class, i.e., $\tilde{Y} \in \mathcal{C} := \{C_1, C_2, C_3\}$ (1000 of each, unknown). In the meanwhile, a data set (X, Y) that only contains 500 instances from each of $\mathcal{C}$ and 5000 images from each of the other classes in $\mathcal{Y} \setminus \mathcal{C}$ is available with all known labels Y .

In this scenario, the training data is imbalanced and only contains a small amount of data that shares the same labels as the target data set. Bayesian optimization methods that only focus on improving the posterior predictive performance on the training data set may find a set of hyperparameters that can only work well on the training data but not necessarily on the target data set. On the other hand, GOI incorporates the expected prediction loss by setting it as its objective function h . It identifies the next promising hyperparameter by evaluating its potential influence on the predictive loss while leveraging it for training performance. Figure 4 shows that while GOI can identify the hyperparameters that lower the prediction performance on target data, other Bayesian optimization methods choose the hyperparameters that may deteriorate the prediction performance.

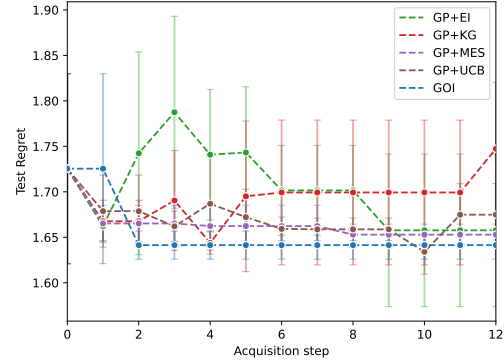


Figure 4: **GOI selects the hyperparameters that provide the best prediction performance on the target test set compared to Bayesian optimization methods.** The hyperparameter optimization is done on a Preactivation Residual Neural Network to predict on a target data set where the unknown labels fall into $\{C_1, C_2, C_3\}$ while the model is trained on a training set, only 4% of which are with labels from the same label set.

5 Discussion

In this work, we propose Goal-Oriented Influence-Based Data Acquisition, a class of effective iterative data acquisition methods, which can be flexibly applied for solving a wide range of problems by defining a task-specific goal objective function. Having the agency to acquire data points, Active Modeling algorithms can substantially reduce the total required data points and enable more efficient and cost-effective model training and problem-solving.

At each iteration, Goal-Oriented Influence acquisition acquires an individual data point and immediately updates the model, which comes at cost of prolonged training time. By acquiring the top b data points that give the lowest individual influence function values, the algorithm can be sped up but at the cost of decreased test accuracy. Independent selection of a batch of data points can lead to data inefficiency due to the correlation between the data points [38]. Future work includes taking into account the mutual information between a set of points and obtaining the most influential acquisition batch with further efficient implementation.

Goal-Oriented Influence uses supervised learning models as the surrogate model. It uses the existing data to calibrate the current state of the knowledge, which is then used to provide insights for the response variable of potential data points as well as target outcomes. The effectiveness of Goal-Oriented Influence in return relies on the accuracy of the predicted values of these “unseen” data points. Given the vast information ready to be subtracted from the candidate training data points as well as the target data points, combining semi-supervised learning techniques could provide more information about the underlying structure of the feature space and therefore potentially improve these predictions. We leave a semi-supervised extension of Goal-Oriented Influence for future work.

References

- [1] Moloud Abdar, Farhad Pourpanah, Sadiq Hussain, Dana Rezazadegan, Li Liu, Mohammad Ghavamzadeh, Paul Fieguth, Xiaochun Cao, Abbas Khosravi, U Rajendra Acharya, et al. A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information Fusion*, 2021.
- [2] Naman Agarwal, Brian Bullins, and Elad Hazan. Second-order stochastic optimization for machine learning in linear time. *The Journal of Machine Learning Research*, 18(1):4148–4187, 2017.
- [3] Ahmed Alaa and Mihaela Van Der Schaar. Discriminative jackknife: Quantifying uncertainty in deep learning via higher-order influence functions. In *International Conference on Machine Learning*, pages 165–174. PMLR, 2020.

- [4] Julyan Arbel, Konstantinos Pitas, Mariia Vladimirova, and Vincent Fortuin. A primer on bayesian neural networks: Review and debates, 2023.
- [5] Jordan Ash, Surbhi Goel, Akshay Krishnamurthy, and Sham Kakade. Gone fishing: Neural active learning with fisher embeddings. In *Advances in neural information processing systems*, volume 34, 2021.
- [6] Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse, uncertain gradient lower bounds. In *International Conference on Learning Representations*, 2020.
- [7] Samyadeep Basu, Philip Pope, and Soheil Feizi. Influence functions in deep learning are fragile. *arXiv*, abs/2006.14651, 2020.
- [8] Samyadeep Basu, Xuchen You, and Soheil Feizi. On second-order group influence functions for black-box predictions. In *International Conference on Machine Learning*, pages 715–724. PMLR, 2020.
- [9] Sanjib Basu, Sreenivasa Rao Jammalamadaka, and Wei Liu. Local posterior robustness with parametric priors: maximum and average sensitivity. In *Maximum Entropy and Bayesian Methods*, pages 97–106. Springer, 1996.
- [10] William H Beluch, Tim Genewein, Andreas Nürnberger, and Jan M Köhler. The power of ensembles for active learning in image classification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 9368–9377, 2018.
- [11] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Proceedings of the 25th International Conference on Neural Information Processing Systems, NIPS’11*, page 2546–2554, Red Hook, NY, USA, 2011. Curran Associates Inc.
- [12] Derek Bingham. Optimization test functions. <http://www.sfu.ca/~ssurjano/ackley.html>, 2015.
- [13] Mickaël Binois and Nathan Wycoff. A survey on high-dimensional gaussian process modeling with application to bayesian optimization. *ACM Trans. Evol. Learn. Optim.*, 2(2), 2022.
- [14] Michael Bloodgood. Support vector machine active learning algorithms with query-by-committee versus closest-to-hyperplane selection. In *IEEE International Conference on Semantic Computing*, pages 148–155, 2018.
- [15] A. J. Booker, J. E. Dennis, P. D. Frank, D. B. Serafini, V. Torczon, and M. W. Trosset. A rigorous framework for optimization of expensive functions by surrogates. *Structural optimization*, 17:1–13, 1999.
- [16] Eric Brochu, Vlad M. Cora, and Nando de Freitas. A tutorial on bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. *arXiv*, abs/1012.2599, 2010.
- [17] Marc-Etienne Brunet, Colleen Alkalay-Houlihan, Ashton Anderson, and Richard Zemel. Understanding the origins of bias in word embeddings. In *International Conference on Machine Learning*, pages 803–811. PMLR, 2019.
- [18] S. Chakraborty, V. Balasubramanian, Q. Sun, S. Panchanathan, and J. Ye. Active batch selection via convex relaxations with guaranteed solution bounds. *IEEE Trans. Pattern Anal. Mach. Intell.*, 37:1945–1958, 2015.
- [19] Gui Citovsky, Giulia DeSalvo, Claudio Gentile, Lazaros Karydas, Anand Rajagopalan, Afshin Rostamizadeh, and Sanjiv Kumar. Batch active learning at scale. In *Proceedings of Advances in Neural Information Processing Systems*, page 11933–11944, 2021.
- [20] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and André van Schaik. Emnist: an extension of mnist to handwritten letters. *arXiv preprint arXiv:1702.05373*, 2017.
- [21] Cody Coleman, Edward Chou, Julian Katz-Samuels, Sean Culatana, Peter Bailis, Alexander C. Berg, Robert Nowak, and I. Zeki Yalniz Roshan Sumbaly, Matei Zaharia. Similarity search for efficient active learning and search of rare concepts. In *Proceedings of the AAAI Conference on Artificial Intelligence*, page 804–812, 2022.
- [22] R Dennis Cook. Detection of influential observation in linear regression. *Technometrics*, 19(1):15–18, 1977.
- [23] R Dennis Cook. Assessment of local influence. *Journal of the Royal Statistical Society: Series B (Methodological)*, 48(2):133–155, 1986.
- [24] R Dennis Cook and Sanford Weisberg. Characterizations of an empirical influence function for detecting influential cases in regression. *Technometrics*, 22(4):495–508, 1980.
- [25] R. Dennis Cook and Sanford Weisberg. *Residuals and influence in regression*. Monographs on statistics and applied probability. Chapman and Hall, New York, 1982.
- [26] P. Donmez, J. Carbonell, and P. N. Bennett. Dual strategy active learning. In *Proceedings of European Conference on Machine Learning*, page 116–127, 2007.

- [27] Bradley Efron. Frequentist accuracy of bayesian estimates. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 77(3):617–646, 2015.
- [28] David Eriksson, Michael Pearce, Jacob R. Gardner, Ryan Turner, and Matthias Poloczek. Scalable global optimization via local bayesian optimization. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, volume 493, page 5496–5507, 2019.
- [29] David Eriksson and Matthias Poloczek. Scalable constrained bayesian optimization. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 130. PMLR, 2021.
- [30] Vitaly Feldman and Chiyuan Zhang. What neural networks memorize and why: Discovering the long tail via influence estimation. *arXiv preprint*, abs/2008.03703, 2020.
- [31] Stanislav Fort, Huiyi Hu, and Balaji Lakshminarayanan. Deep ensembles: A loss landscape perspective. *arXiv*, abs/1912.02757, 2020.
- [32] P. Frazier. A tutorial on bayesian optimization. *ArXiv*, abs/1807.02811, 2018.
- [33] Peter I. Frazier, Warren B. Powell, and Savas Dayanik. A knowledge-gradient policy for sequential information collection. *SIAM J. Control Optim.*, 47(5):2410–2439, 2008.
- [34] Peter I. Frazier, Warren B. Powell, and Savas Dayanik. The knowledge-gradient policy for correlated normal beliefs. *INFORMS Journal on Computing*, 21(4):599–613, 2009.
- [35] Alexander Freytag, Erik Rodner, and Joachim Denzler. Selecting influential examples: Active learning with expected model output changes. In *Computer Vision - ECCV*, volume 8692, pages 562–577, 2014.
- [36] Lukas P. Fröhlich, Edgar D. Klenske, Julia Vinogradskaya, Christian Daniel, and Melanie N. Zeilinger. Noisy-input entropy search for efficient robust bayesian optimization. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics*, volume 108, 2020.
- [37] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1050–1059, New York, New York, USA, 20–22 Jun 2016. PMLR.
- [38] Yarin Gal, Riashat Islam, and Zoubin Ghahramani. Deep bayesian active learning with image data. In *International Conference on Machine Learning*, pages 1183–1192. PMLR, 2017.
- [39] Roman Garnett. *Bayesian Optimization*. Cambridge University Press, 2023.
- [40] Y. Geifman and R. El-Yaniv. Deep active learning with a neural architecture search. In *Proceedings of Advances in Neural Information Processing Systems*, page 5974–5984, 2019.
- [41] Ryan Giordano, Tamara Broderick, and Michael I Jordan. Covariances, robustness, and variational bayes. *arXiv preprint*, abs/1709.02536, 2017.
- [42] B. Gu, Z. Zhai, C. Deng, and H. Huang. Efficient active learning by querying discriminative and representative samples and fully exploiting unlabeled data. *IEEE Trans. Neural Netw. Learn. Syst.*, 32:4111–4122, 2021.
- [43] D. Gudovskiy, A. Hodgkinson, T. Yamaguchi, and S. Tsukizawa. Deep active learning for biased datasets via fisher kernel self-supervision. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 9038–9046, 2020.
- [44] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70, ICML’17*, page 1321–1330. JMLR.org, 2017.
- [45] Han Guo, Nazneen Fatema Rajani, Peter Hase, Mohit Bansal, and Caiming Xiong. Fastif: Scalable influence functions for efficient model interpretation and debugging. *arXiv preprint*, abs/2012.15781, 2020.
- [46] Paul Gustafson et al. Local sensitivity of posterior expectations. *Annals of statistics*, 24(1):174–195, 1996.
- [47] Xiaochuang Han, Byron C Wallace, and Yulia Tsvetkov. Explaining black box predictions and unveiling data artifacts through influence functions. *arXiv preprint*, abs/2005.06676, 2020.
- [48] M. Hasan, S. Paul, A. I. Mourikis, and A. K. Roy-Chowdhury. Context-aware query selection for active learning in event recognition. *IEEE Trans. Pattern Anal. Mach. Intell.*, 42:554–567, 2020.
- [49] Kaiming He, X. Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *European Conference on Computer Vision*, 2016.
- [50] Philipp Hennig and Christian J. Schuler. Entropy search for information-efficient global optimization. *Journal of Machine Learning Research*, 13(6):1809–1837, 2012.

- [51] Neil Houlsby, Ferenc Huszár, Zoubin Ghahramani, and Máté Lengyel. Bayesian active learning for classification and preference learning. *ArXiv*, abs/1112.5745, 2011.
- [52] Neil Houlsby, Ferenc Huszár, Zoubin Ghahramani, and Máté Lengyel. Bayesian active learning for classification and preference learning. *ArXiv*, abs/1112.5745, 2011.
- [53] S. Huang, T. Wang, H. Xiong, J. Huan, and D. Dou. Semi-supervised active learning with temporal output discrepancy. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, pages 3447–3456, 2021.
- [54] Peter J. Huber. The 1972 wald lecture robust statistics: A review. *The Annals of Mathematical Statistics*, 43(4):1041–1067, 1972.
- [55] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *Learning and Intelligent Optimization*, pages 507–523, 2011.
- [56] Q. Jin, M. Yuan, Q. Qiao, and Z. Song. One-shot active learning for image segmentation via contrastive learning and diversity-based sampling. *Knowl.-Based Syst.*, 241, 2022.
- [57] Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998.
- [58] Kaggle. Sentiment analysis on movie reviews: Classify the sentiment of sentences from the rotten tomatoes dataset. <https://www.kaggle.com/c/sentiment-analysis-on-movie-reviews/data>, 2015.
- [59] Samuel Kim, Peter Y. Lu, Charlotte Loh, Jamie Smith, Jasper Snoek, and Marin Soljačić. Deep learning for bayesian optimization of scientific problems with high-dimensional structure. *arXiv*, abs/2104.11667, 2022.
- [60] Y. Kim and B. Shin. In defense of core-set: A density-aware coreset selection for active learning. In *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, page 804–812, 2022.
- [61] Yoon-Yeong Kim, Kyungwoo Song, JoonHo Jang, and Il-chul Moon. Lada: Look-ahead data acquisition via augmentation for deep active learning. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 22919–22930. Curran Associates, Inc., 2021.
- [62] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [63] Andreas Kirsch, Joost van Amersfoort, and Yarin Gal. Batchbald: Efficient and diverse batch acquisition for deep bayesian active learning. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [64] Pang Wei Koh, Kai-Siang Ang, Hubert HK Teo, and Percy Liang. On the accuracy of influence functions for measuring group effects. *arXiv preprint*, abs:1905.13289, 2019.
- [65] Pang Wei Koh and Percy Liang. Understanding black-box predictions via influence functions. *arXiv preprint*, abs/1703.04730, 2017.
- [66] Alex Krizhevsky. Learning multiple layers of features from tiny images, 2009.
- [67] Harold J. Kushner. A new method of locating the maximum point of an arbitrary multipeak curve in the presence of noise. *Journal of Basic Engineering*, 86(1):97–106, 1964.
- [68] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS’17, page 6405–6416, 2017.
- [69] Yann LeCun, Corinna Cortes, and Christopher C. J. Burges. The mnist handwritten digit database. <http://yann.lecun.com/exdb/mnist/>, 1998.
- [70] Jeremy Lewi, Robert Butera, and Liam Paninski. Efficient active learning with generalized linear models. In *Artificial Intelligence and Statistics*, pages 267–274, 2007.
- [71] C. Li, H. Ma, Z. Kang, Y. Yuan, X.-Y. Zhang, , and G. Wang. On deep unsupervised active learning. In *Proc. 29th Int. Joint Conf. Artif. Intell.*, page 2626–2632, 2020.
- [72] Dongyuan Li, Zhen Wang, Yankai Chen, Renhe Jiang, Weiping Ding, and Manabu Okumura. A survey on deep active learning: Recent advances and new frontiers. *IEEE Transactions on Neural Networks and Learning Systems*, 36(4):5879–5899, 2025.
- [73] S. Li, J. M. Phillips, X. Yu, R. M. Kirby, and S. Zhe. Batch multifidelity active learning with budget constraints. In *Proceedings of Advances in Neural Information Processing Systems*, page 995–1007, 2022.

- [74] W. Li, G. Dasarathy, K. N. Ramamurthy, and V. Berisha. Finding the homology of decision boundaries with active learning. In *Advances in Neural Information Processing Systems*, page 8355–8365, 2020.
- [75] Yucen Lily Li, Tim G. J. Rudner, and Andrew Gordon Wilson. A study of bayesian neural network surrogates for bayesian optimization. In *The Twelfth International Conference on Learning Representations*, 2024.
- [76] Zhuoming Liu, Hao Ding, Huaping Zhong, Weijia Li, Jifeng Dai, and Conghui He. Influence selection for active learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [77] Turab Lookman, Prasanna V. Balachandran, Dezhen Xue, and Ruihao Yuan. Active learning in materials science with emphasis on adaptive sampling using uncertainties for targeted design. *npj Computational Materials*, 5(1):1–17, 2019.
- [78] James Martens. Deep learning via hessian-free optimization. In *Proceedings of the 27th International Conference on International Conference on Machine Learning*, page 735–742, 2010.
- [79] R.K. Mehra. Computation of the inverse hessian matrix using conjugate gradient methods. *Proceedings of the IEEE*, 57(2):225–226, 1969.
- [80] Jonas Moćkus. On bayesian methods for seeking the extremum and their application. In *Optimization Techniques IFIP Technical Conference Novosibirsk*, pages 400–404. Springer Berlin Heidelberg, 1975.
- [81] Amin Nayebe, Alexander Munteanu, and Matthias Poloczek. A framework for Bayesian optimization in embedded subspaces. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 4752–4761. PMLR, 2019.
- [82] ChangYong Oh, Efstratios Gavves, and Max Welling. BOCK : Bayesian optimization with cylindrical kernels. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 3868–3877. PMLR, 2018.
- [83] Bo Pang and Lillian Lee. Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. In *Proceedings of the 43rd Annual Meeting of the ACL*, page 115–124, 2005.
- [84] Liam Paninski. Asymptotic theory of information-theoretic experimental design. *Neural Computation*, 17(7):1480–1507, 07 2005.
- [85] A. Parvaneh, E. Abbasnejad, D. Teney, R. Haffari, A. Van Den Hengel, and J. Q. Shi. Active learning by feature mixing. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, page 12227–12236, 2022.
- [86] Victor Picheny, Tobias Wagner, and David Ginsbourger. A benchmark of kriging-based infill criteria for noisy optimization. *Structural and Multidisciplinary Optimization*, 48, 2013.
- [87] Ghanshyam Paliania. Machine learning in materials science: From explainable predictions to autonomous design. *Computational Materials Science*, 193:110360, 2021.
- [88] Hiranmayi Ranganathan, Hemanth Venkateswara, Shayok Chakraborty, and Sethuraman Panchanathan. Deep active learning for image classification. In *IEEE International Conference on Image Processing*, pages 3934–3938, 2017.
- [89] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- [90] Rommel G. Regis and Christine A. Shoemaker. Improved strategies for radial basis function methods for global optimization. *J. of Global Optimization*, 37(1):113–135, January 2007.
- [91] Rommel G. Regis and Christine A. Shoemaker. Parallel radial basis function methods for the global optimization of expensive functions. *European Journal of Operational Research*, 182(2):514–535, 2007.
- [92] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xin Wang. A survey of deep active learning. *ACM Comput. Surv.*, 54(9), October 2021.
- [93] D. Roth and K. Small. Margin-based active learning for structured output spaces. In *Proc. Eur. Conf. Mach. Learn.*, page 413–424, 2006.
- [94] Nicholas Roy and Andrew McCallum. Toward optimal active learning through monte carlo estimation of error reduction. In *International Conference on Machine Learning*, page 441–448, 2001.
- [95] Akanksha Saran, Safoora Yousefi, Akshay Krishnamurthy, John Langford, and Jordan T. Ash. Streaming active learning with deep neural networks. *arXiv*, abs/2303.02535, 2023.
- [96] Greg Schohn and David Cohn. Less is more: Active learning with support vector machines. In *Proceedings of the Seventeenth International Conference on Machine Learning*, pages 839–846, 2000.

- [97] Christopher Schröder and Andreas Niekler. A survey of active learning for text classification using deep neural networks. *ArXiv*, abs/2008.07267, 2020.
- [98] Warren Scott, Peter Frazier, and Warren Powell. The correlated knowledge gradient for simulation optimization of continuous parameters using gaussian process regression. *SIAM Journal on Optimization*, 21(3):996–1026, 2011.
- [99] Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach. In *International Conference on Learning Representations*, 2018.
- [100] Burr Settles. Active learning literature survey. In *University of Wisconsin, Madison*, 2009.
- [101] Burr Settles, Mark Craven, and Soumya Ray. Multiple-instance active learning. In John C. Platt, Daphne Koller, Yoram Singer, and Sam T. Roweis, editors, *Advances in Neural Information Processing Systems 20*, pages 1289–1296, 2007.
- [102] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.
- [103] Yihang Shen and Carl Kingsford. Computationally efficient high-dimensional bayesian optimization via variable selection. In *Proceedings of the Second International Conference on Automated Machine Learning*, volume 224, pages 1–27, 2023.
- [104] Changjian Shui, Fan Zhou, Christian Gagné, and Boyu Wang. Deep active learning: Unified and principled method for query and training. In Silvia Chiappa and Roberto Calandra, editors, *The 23rd International Conference on Artificial Intelligence and Statistics*, volume 108 of *Proceedings of Machine Learning Research*, pages 1308–1318, 2020.
- [105] Aditya Siddhant and Zachary C Lipton. Deep bayesian active learning for natural language processing: Results of a large-scale empirical study. *arXiv preprint*, abs/1808.05697, 2018.
- [106] Samarth Sinha, Sayna Ebrahimi, and Trevor Darrell. Variational adversarial active learning. *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5971–5980, 2019.
- [107] Jasper Snoek, Oren Rippel, Kevin Swersky, Ryan Kiros, Nadathur Satish, Narayanan Sundaram, Md. Mostofa Ali Patwary, Prabhat Prabhata, and Ryan P. Adams. Scalable bayesian optimization using deep neural networks. In *Proceedings of the 32nd International Conference on International Conference on Machine Learning*, volume 37, page 2171–2180. JMLR.org, 2015.
- [108] Jost Tobias Springenberg, Aaron Klein, Stefan Falkner, and Frank Hutter. Bayesian optimization with robust bayesian neural networks. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016.
- [109] N. Srinivas, A. Krause, S. Kakade, and M. Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. In *Proceedings of the 27th International Conference on International Conference on Machine Learning*, pages 1015–1022, 2010.
- [110] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.
- [111] J. Sun, H. Zhai, O. Saisho, and S. Takeuchi. Beam search optimized batch bayesian active learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 6084–6091, 2023.
- [112] Y. Sverchkov and M. Craven. A review of active learning approaches to experimental design for uncovering biological networks. *PLoS computational biology*, 13(6), 2017.
- [113] Michael E. Tipping and Christopher M. Bishop. Probabilistic principal component analysis. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 61(3):611–622, 1999.
- [114] A. Törn and A. Zilinskas. Global optimization, 1989.
- [115] Toan Tran, Thanh-Toan Do, Ian D. Reid, and G. Carneiro. Bayesian generative active deep learning. In *International Conference on Machine Learning*, 2019.
- [116] John Tukey. Bias and confidence in not quite large samples. *Ann. Math. Statist.*, 29:614, 1958.
- [117] D. Wang and Y. Shang. A new active labeling method for deep learning. In *Proc. of IJCNN*, pages 112–119, 2014.
- [118] Keze Wang, Dongyu Zhang, Ya Li, Ruimao Zhang, and Liang Lin. Cost-effective active learning for deep image classification. *IEEE Transactions on Circuits and Systems for Video Technology*, 27(12):2591–2600, 2017.

- [119] Tianyang Wang, Xingjian Li, Pengkun Yang, Guosheng Hu, Xiangrui Zeng, Siyu Huang, Cheng-Zhong Xu, and Min Xu. Boosting active learning via improving test performance. *arXiv*, abs/2112.05683, 2022.
- [120] Xilu Wang, Yaochu Jin, Sebastian Schmitt, and Markus Olhofer. Recent advances in bayesian optimization. *ACM Comput. Surv.*, 55(13s), 2023.
- [121] Zi Wang, Clement Gehring, Pushmeet Kohli, and Stefanie Jegelka. Batched large-scale bayesian optimization in high-dimensional spaces. In *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics*, volume 84 of *Proceedings of Machine Learning Research*, pages 745–754, 2018.
- [122] Zi Wang and Stefanie Jegelka. Max-value entropy search for efficient bayesian optimization. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, page 3627–3635. JMLR.org, 2017.
- [123] Jian Wu and Peter I. Frazier. The parallel knowledge gradient method for batch bayesian optimization. In *Proceedings of the 30th International Conference on Neural Information Processing Systems*, page 3134–3142, Red Hook, NY, USA, 2016.
- [124] Jian Wu, Saul Toscano-Palmerin, P. Frazier, and Andrew Gordon Wilson. Practical multi-fidelity bayesian optimization for hyperparameter tuning. *ArXiv*, abs/1903.04703, 2019.
- [125] Meng Xia and Ricardo Henao. Reliable active learning via influence functions. *Transactions on Machine Learning Research*, 2023.
- [126] Binhui Xie, Longhui Yuan, Shuang Li, Chi Harold Liu, Xinjing Cheng, and Guoren Wang. Active learning for domain adaptation: An energy-based approach. In *Proc. of AAAI*, page 8708–8716, 2022.
- [127] Minjie Xu and Gary Kazantsev. Understanding goal-oriented active learning via influence functions. *arXiv*, abs/1905.13183, 2019.
- [128] Lin Yang, Yizhe Zhang, Jianxu Chen, Siyuan Zhang, and Danny Z Chen. Suggestive annotation: A deep active learning framework for biomedical image segmentation. In *International conference on medical image computing and computer-assisted intervention*, pages 399–407. Springer, 2017.
- [129] Changchang Yin, Buyue Qian, Shilei Cao, Xiaoyu Li, Jishang Wei, Qinghua Zheng, and Ian Davidson. Deep similarity-based batch mode active learning with exploration-exploitation. In Vijay Raghavan, Srinivas Aluru, George Karypis, Lucio Miele, and Xindong Wu, editors, *2017 IEEE International Conference on Data Mining*, pages 575–584, 2017.
- [130] Tong Zhang and Frank J. Oles. The value of unlabeled data for classification problems. In *International Conference on Machine Learning*, volume 20, page 1191–1198, 2000.
- [131] Y. Zhao, Z. Shi, J. Zhang, D. Chen, and L. Gu. A novel active learning framework for classification: Using weighted rank aggregation to achieve multiple query criteria. *Pattern Recognition*, 93:581–602, 2019.
- [132] Fedor Zhdanov. Diverse mini-batch active learning. *arXiv preprint arXiv:1901.05954*, 2019.